



Subject Areas:

Computer Science, Artificial
Intelligence

Keywords:

Large Language Models, Planning,
Code Generation

Author for correspondence:

Mayank Kejriwal

e-mail: kejriwal@isi.edu

Code-Driven Planning in Grid Worlds with Large Language Models

Ashwath Vaithinathan Aravindan¹, Zhisheng Tang^{1,2} and Mayank Kejriwal^{1,2}

¹University of Southern California, Los Angeles, 90007, California, United States of America

²Information Sciences Institute, 4676 Admiralty Way #1001, Los Angeles, 90292, California, United States of America

We propose an iterative programmatic planning (IPP) framework that solves grid based tasks by using large language models (LLMs) to synthesize interpretable, executable programs that act as agent policies mapping environment states to action sequences, rather than relying on traditional search or reinforcement learning. Alongside prompting strategies such as direct code generation, pseudocode conditioned refinement, and curriculum based prompting, IPP centers on an iterative refinement mechanism that updates code from coarse, task-level feedback. We evaluate our approach using six leading LLMs and two challenging grid based benchmarks (GRASP and MiniGrid). Our IPP framework demonstrates improvements over direct code generation for five of the six models, with net-energy gains ranging from a few percent to roughly threefold and, for Gemini-2.5-Pro, a sign flip from a negative baseline to a strongly positive score, establishes a new state of the art result for GRASP, and significantly outperforms direct elicitation from GPT-o3-mini (by 43% on the hardest MiniGrid task to 116% on GRASP). While code generation has a higher initial prompting cost compared to direct solution elicitation (\$0.08 per task vs. \$0.002 per instance for GPT-o3-mini), the code can be reused for any number of instances, making the amortized cost significantly lower (by 400x on GPT-o3-mini across the complete GRASP benchmark). Beyond performance gains, IPP produces reusable, human-readable policies that support transparent inspection and modification, underscoring the advantages of planning as program synthesis over per-instance inference. The code and results can be found [here](#).

1. Introduction

Grid based environments are a core abstraction for planning tasks in robotics [1–3], embodied AI [4], and reinforcement learning (RL) [5–8]. These environments offer a formal, discrete setting to study core aspects of intelligent behavior, such as planning [9,10], exploration [11], and generalization [12]. Their simplicity enables systematic evaluation across diverse approaches: from classical search [13] and reinforcement learning [14,15] to neurosymbolic [16] and language based methods [17]. They are commonly used as a simplified abstraction of domains such as robotics and navigation, capturing spatial and symbolic reasoning while stripping away perception noise and sensor complexity. We treat them not as a stand-in for real-world robotics, which involves continuous, noisy, and only partially observable dynamics, but as a controlled setting in which decision-making and planning can be studied and compared systematically across methods.

Meanwhile, large language models (LLMs) have recently shown strong performance on a variety of reasoning tasks, including question answering [18,19], tool use [20,21], and code generation [22,23]. However, LLMs have struggled with planning [24], which requires state tracking, goal directed search, and consistent multi-step decision-making, all of which LLMs do not naturally exhibit. At the same time, LLMs excel at generating interpretable code [23], which motivates a shift in perspective: instead of prompting LLMs to produce action plans directly, we treat planning as code generation. LLMs can generate executable programs that act as action policies. This capability aligns *planning with programming*, since both require symbolic reasoning over structured inputs and constraints. Additionally, such an approach offers several advantages: generated programs are interpretable, reusable across task instances, and cheap to execute.

Compared to RL, which relies on trial and error interaction and costly gradient based optimization, a code-driven approach offers a lightweight and interpretable alternative that requires no task specific training. While RL excels in grid based domains, it often suffers from poor generalization [25,26] and produces opaque policies [8,27]. In contrast, a code-driven approach replaces policy learning with LLM-based program synthesis, enabling the generation of symbolic and reusable policies that are efficient and transparent, and aligning with a broader shift toward grounding LLMs in executable environments [28–30] where code serves as a verifiable and editable interface between high-level reasoning and low-level control. For the class of problems we consider, namely planning with deterministic, fully observable dynamics that are known in advance, generated code can serve as a lightweight and practical alternative to RL for building generalizable planning agents. Settings with stochastic transitions, partial observability, or dynamics that must be learned through interaction still call for RL or other methods that explicitly reason about uncertainty.

In this work, we propose **Iterative Programmatic Planning (IPP)**, a code-driven framework that uses LLMs to generate Python programs as action policies. IPP incorporates an **Iterative Refinement (IR)** loop, where LLM-generated programs are executed, evaluated, and revised based on feedback to correct errors and improve task performance. Iterative self-refinement, in which a model revises its own outputs in response to feedback, is by now a well-established idea in the LLM literature [31,32]. What is new in IPP is how this idea is instantiated for planning: the refined artifact is a single, reusable policy program rather than a natural language answer or chain of thought, and refinement is driven solely by a coarse, task-level signal such as reward rather than fine-grained or per-step supervision. Studying this setting across six models and two benchmarks, we find that such coarse feedback is often sufficient to iteratively synthesize policies that match or exceed strong baselines. This also sets IPP apart from prior code-as-policy systems such as Voyager [33], which incrementally grows a library of skills through environment interaction, and WorldCoder [34], which synthesizes a code-based model of the environment’s dynamics; IPP maintains no skill library and learns no world model, treating the policy program itself as the object to be iteratively improved. Moreover, whereas Voyager keeps an LLM in the loop throughout its lifetime to propose tasks and author skills, IPP invokes an LLM only during a

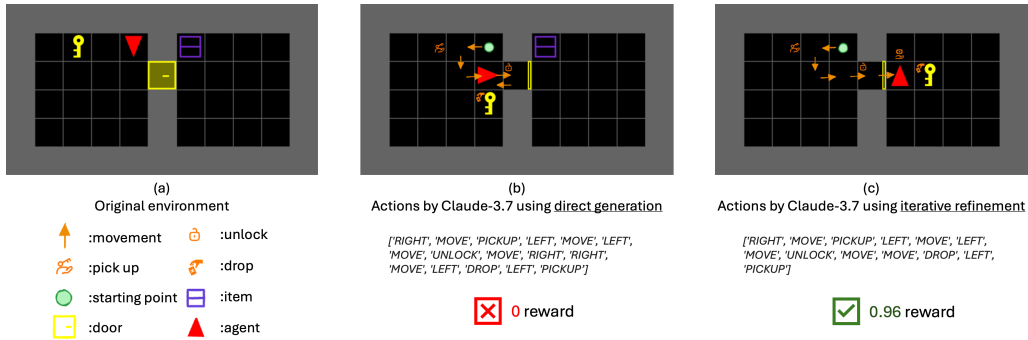


Figure 1: Example of the *Unlock Pickup* task from the MiniGrid benchmark. The agent (red triangle) must retrieve a box (purple item) located in a separate room behind a locked door (yellow). Directions in the action sequence indicate the direction the agent turns to face before moving. (a) shows the original environment setup (b) shows actions generated by Claude-Sonnet-3.7 using *direct generation*, which fails and yields a reward of 0. (c) shows *iterative refinement*, which successfully completes the task with a reward of 0.96.

bounded refinement phase, after which the synthesized program is executed on any number of task instances without further LLM calls.

Figure 1 illustrates one example of performance improvement enabled by iterative refinement. We evaluate IPP on two grid based benchmarks: GRASP [35] and MiniGrid [7], across six state of the art LLMs, and show that it produces robust and generalizable *code-based policies*. Compared to directly prompting for action sequence and other baselines, IPP offers better performance, greater transparency, and lower computational cost. In summary, this work makes the following contributions:

- We propose Iterative Programmatic Planning (IPP), a novel framework that treats planning as code generation. Instead of producing action sequences directly, LLMs synthesize interpretable Python programs as reusable policies for grid based environments. IPP introduces an *Iterative Refinement* process that improves these programs based on task-specific feedback. This approach offers transparency, efficiency, and generalization without requiring task-specific training.
- We evaluate IPP on two benchmarks: GRASP and MiniGrid, across six leading LLMs. IPP improves the average net energy over direct code generation on GRASP for five of the six models (for example, a threefold increase for GPT-o1, from 0.94 to 2.82, and turning Gemini-2.5-Pro's negative baseline into a strongly positive score, from -0.52 to 3.22), and raises GPT-o3-mini's reward on the hardest MiniGrid task, Unlock-Pickup, from 0.00 to 0.78. Compared to non-code-based CoT and 2-step CoT prompting, IPP attains equal or better performance at significantly lower cost, since its synthesized policies generalize without requiring per-instance inference.
- We introduce two structured prompting strategies: *pseudocode Extension*, where LLMs turn a high-level algorithmic sketch into executable code, and *step-by-step curriculum prompting*, where the complexity of the task is incrementally added. In the GRASP benchmark, pseudocode extension improves energy collection for Claude-Sonnet-3.7 from 0.89 (Greedy) to 0.98 (+10%), and for GPT-o3-mini to 1.36 (+52%). Using step-by-step curriculum prompting, GPT-o3-mini achieves steady gains — from 0.44 to 1.73 (+293%) at final complexity — while Claude-Sonnet-3.7 improves from 0.12 to 1.42 (+1083%) from Step 0 to Step 2 before leveling off, demonstrating the benefit of curriculum based synthesis.

2. Preliminaries

(a) Code as Policy

Large Language Models (LLMs) have demonstrated strong capabilities in generating executable and interpretable code across a variety of domains. Hence, in RL settings, they have been explored for use as policy synthesizers [36,37]. The code itself acts as a policy, as a medium to define conditional logic, loops, and decision structures that generalize beyond a single instance.

(b) In-Context Learning

LLMs exhibit an ability to condition their output on examples, task descriptions, or structured prompts presented within its input context, known as in-context learning (ICL) [38]. More specifically, in our work, we utilize a form of ICL known as Few-Shot Prompting [39], by providing a small number of input–output examples within the prompt to guide the model’s behavior on new, unseen instances. Although it is generally used to communicate task structure, format, and reasoning patterns, we leverage this capability for policy refinement in the proposed Iterative Programmatic Planning (IPP) framework.

(c) Planning Problems

We consider planning problems that can be represented as discrete, grid based environments. We assume that each environment is fully observable and composed of cells, and each cell is associated with attributes such as obstacles, items, and goal relevant entities. Agents must navigate these grids and execute a task e.g., item collection or goal reaching—by performing valid action sequences that satisfy task specific constraints. These environments are common in domains like embodied AI, robotics, and reinforcement learning [2,4,7].

Formally, each planning problem is represented as a tuple $\mathcal{P} = (\mathcal{S}, \mathcal{A}, T, s_0, \mathcal{G})$, where $\mathcal{S}$ is the state space; each $s \in \mathcal{S}$ denotes a configuration of the grid environment, including the agent’s position and attributes (e.g., energy, inventory), as well as locations of objects and obstacles. $\mathcal{A}$ is the finite set of discrete actions (e.g., move, pick up, unlock, drop) that the agent is allowed. $T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is the predefined transition function. We assume the transition function is deterministic and fully defined, so the effects of each action in a given state are predictable and known. $s_0 \in \mathcal{S}$ is the initial state. We assume that this is fully observable, meaning the agent has access to the complete initial state of the environment. Finally, $\mathcal{G} \subseteq \mathcal{S}$ is the set of goal states.

A valid plan is then defined as a sequence of actions $\pi = [a_0, a_1, \dots, a_T]$ such that the resulting state after executing π from s_0 falls within the goal region: $T(T(\dots T(s_0, a_0), a_1), \dots, a_T) \in \mathcal{G}$. The agent’s task is to generate a sequence of actions $\pi = [a_0, a_1, \dots, a_T]$ that transforms the initial state $s_0 \in \mathcal{S}$ into a goal state $s_T \in \mathcal{G}$ under the transition function T . This requires reasoning about the structure of the environment, anticipating the effects of actions via $T(s, a)$, and selecting actions from $\mathcal{A}$ that satisfy task specific constraints while getting closer toward the objective.

3. Approach

To solve such planning problems, we propose an Iterative Programmatic Planning (IPP) framework that uses LLMs to synthesize interpretable and executable programs that serve as action policies. In this framework, the LLM generates programs that map environment observations to sequences of allowable actions, effectively functioning as planning agents. We first decompose the problem into three key components: (1) *direct program synthesis*, where the LLM is prompted to generate complete, executable programs in a general purpose interpretable language (e.g., Python) from natural language task descriptions; (2) *pseudocode conditioned refinement*, where the model is seeded with a high level algorithmic sketch (e.g., greedy search) and tasked with completing or adapting

it to the task environment; and (3) *curriculum based prompting*, where the model is guided through a sequence of incrementally complex subproblems to facilitate generalization to harder tasks.

We combine these components into a unified prompting¹ based approach suitable for zero shot and few shot planning through code generation. However, to further *optimize* the program generated by the model, we also incorporate an iterative refinement process whereby the LLM first generates an initial candidate program, which is then executed and evaluated on a set of task instances. Based on some performance metric (such as reward), the model is prompted to revise the code. Iteration continues until performance converges or improves no further. Through refinement, we show that the LLM learns to correct logical errors, incorporate missing constraints, and restructure control flow.

(a) Code-driven Planning with Iterative Refinement

Rather than relying on traditional search algorithms or reinforcement learning to solve $\mathcal{P}$, code as plan frameworks use an LLM to generate an interpretable program f that, when executed against a specific environment instance, produces a sequence of actions $\pi = [a_0, a_1, \dots, a_T]$ transitioning the agent from the initial state s_0 to a goal state $s_T \in \mathcal{G}$. This program acts as a reusable and interpretable policy that maps environment observations to action sequences.

This approach contrasts with classical reinforcement learning methods, which typically learn a policy $\pi: \mathcal{S} \rightarrow \mathcal{A}$ through repeated interaction with the environment and optimization of cumulative rewards, a process that is time consuming and computationally intensive. At the other extreme, an LLM could be prompted to directly produce a plan given a grid based planning problem. Experimentally, this approach is found to yield significantly low quality solutions, even for leading LLMs (Section 5; see also Figure 1). Code-driven planning offers a compromise by forcing the LLM to provide an explicit policy, using only a few prompts, encoded as an interpretable program. An auxiliary advantage of such a policy is that its logic is observable and interpretable.

Direct Generation Our baseline approach, *direct generation*, prompts the LLM to directly synthesize a complete and executable program f given only a natural language task description and environment specifications. This approach assumes that the model can generate code that considers both a high level strategy and low level execution details in a single pass. While fast and straightforward, this approach may overlook edge cases or misapply rules. Nonetheless, it establishes a strong starting point for comparing more advanced techniques. Decomposing a complex task into a planning phase followed by an execution phase has been shown to improve reasoning quality in large language models [40]. Accordingly, as a Step 1, we prompt the model to generate only the algorithm's pseudocode. After receiving this intermediate output, we proceed to Step 2, in which the model generates the final code according to a fixed function API. This two-stage procedure not only improves code correctness but also provides greater interpretability, as the intermediate planning step reveals the model's reasoning before code synthesis.

Pseudocode Extension To address some of the flaws of direct generation, we introduce a pseudocode based prompting strategy. Here, the LLM is provided with a high level baseline method (e.g., greedy or rule based logic in pseudocode) and tasked with extending and refining it into an executable program f . This approach narrows the LLM's search space, allowing the model to use *symbolic priors* while preserving flexibility. A disadvantage of this approach is that it requires the pseudocode prior to be provided in the first place. For complex grids, providing such a prior may be cumbersome or even infeasible.

Step-by-Step The *step-by-step* approach introduces a curriculum style procedure, in which the model is prompted to solve increasingly complex versions of the task, gradually building up to the full problem. This curriculum design follows the same least-to-most prompting principle

¹ All prompt templates for the methods described in this section are provided in [i](#), [ii](#), [iii](#) and [iv](#).

of decomposing a hard problem into a sequence of simpler subproblems [40]. The step-by-step approach is purely prompting based: at each step it is asked to update its code to accommodate one additional parameter or constraint, building up to the full task. Beginning with simplified environments (e.g., no obstacles, no step cost), the model produces initial code $f^{(0)}$ that works under minimal assumptions. At each subsequent stage i , the model is given an updated problem with more complexity, such as limited energy, obstacles, or carry constraints, and is required to revise and extend the previous program $f^{(i-1)}$ into a new program $f^{(i)}$ that can solve the current problem. This formulation mimics human incremental reasoning and in theory, should help the LLM produce a strategy that it might fail to produce through (for example) one pass direct generation.

Iterative Refinement To improve the correctness and generality of the generated program f , we propose an *Iterative Refinement (IR)* strategy, whereby an LLM iteratively improves its generated code through alternating steps of evaluation and feedback driven refinement (Algorithm 1, see also the flowchart in Figure 2). Starting from an initial candidate program $f^{(0)}$ (obtained via direct generation), we evaluate its performance on a training set of planning problems $\mathcal{P}$ and measure its performance via a predefined task specific metric $J^{(i)}$ for each instance.

The aggregate performance $J_{\text{curr}} = \text{Aggregate}(J^{(i)})$ is then compared to the previous iteration's performance J_{prev} . If no improvement is observed, we assume convergence and terminate the loop, returning the previous program $f^{(t-1)}$, i.e., the best-performing iterate. Otherwise, we identify the k worst performing instances (i.e., those with the lowest $J^{(i)}$ values) as $\mathcal{D}_{\text{fail}}$, and prompt the LLM to refine the current program $f^{(t)}$ based on this feedback. The revised program $f^{(t+1)}$ is expected to address the identified issues while preserving correct behavior elsewhere.

From an optimization perspective, IR can be viewed as a discrete, non gradient based search procedure over the space of executable programs. Each refinement step attempts to improve a program $f^{(t)}$ by minimizing an implicit loss function, which is captured via task specific evaluation metrics, over a finite dataset of grid instances. The feedback driven update acts as a heuristic gradient, guiding the LLM to generate a new program $f^{(t+1)}$ that corrects previous failures while maintaining performance on successful instances. While analytically not an optimizer, IR approximates iterative function minimization in a symbolic space, guided by performance signals rather than gradients. In contrast to the step-by-step strategy, iterative refinement gives the model all of the problem's constraints from the outset and improves the code based specifically on the instances where it fails.

4. Experimental setup

We evaluate our proposed framework on two benchmarks: the GRASP dataset [35] and three tasks from MiniGrid [7]. They are chosen for their structured 2D grid environments, which can be easily converted into LLM prompts (Appendix (c)).

Algorithm 1 Iterative Refinement

Require: Planning problem $\mathcal{P} = (\mathcal{S}, \mathcal{A}, T, s_0, \mathcal{G})$

- 1: $f^{(0)} \leftarrow \text{DirectGeneration}(\mathcal{P})$
- 2: $J_{\text{prev}} \leftarrow -\infty$
- 3: $t \leftarrow 0$
- 4: **while** True **do**
- 5: $\mathcal{D} \leftarrow$ evaluation results of $f^{(t)}$ on training set
- 6: $J_{\text{curr}} \leftarrow$ average over all $J^{(i)}$ in $\mathcal{D}$
- 7: **if** $J_{\text{curr}} \leq J_{\text{prev}}$ **then**
- 8: **break**
- 9: **end if**
- 10: $J_{\text{prev}} \leftarrow J_{\text{curr}}$
- 11: $\mathcal{D}_{\text{fail}} \leftarrow \text{SelectWorstCases}(\mathcal{D}, k=3)$
- 12: $f^{(t+1)} \leftarrow \text{RefineWithFeedback}(f^{(t)}, \mathcal{D}_{\text{fail}})$
- 13: $t \leftarrow t + 1$
- 14: **end while**
- 15: **return** $f^{(t-1)}$

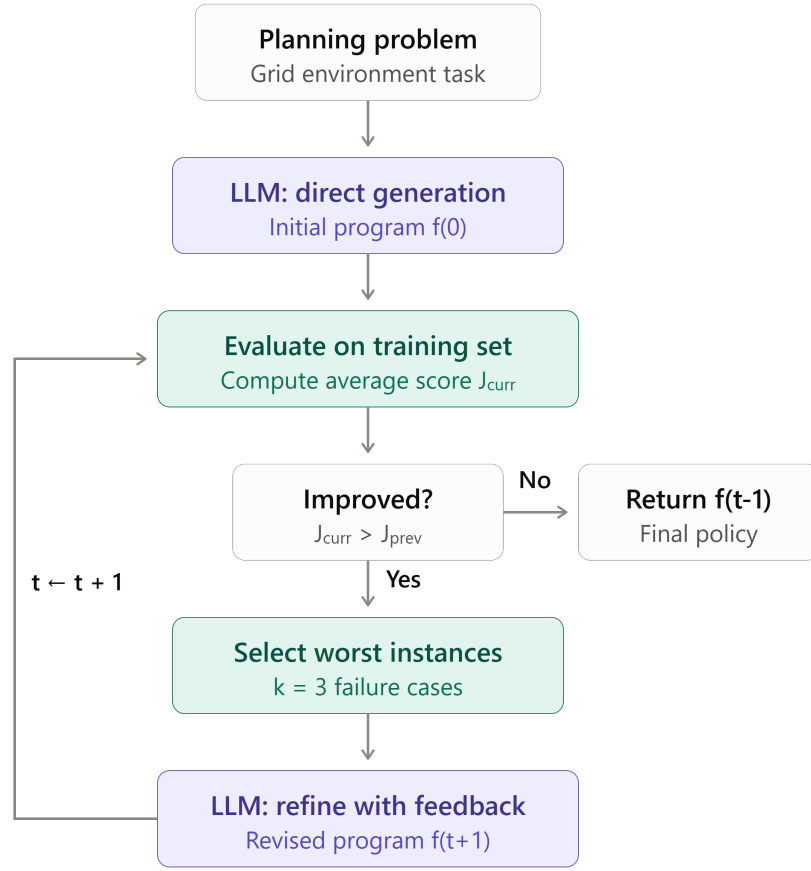


Figure 2: Flowchart of the Iterative Refinement (IR) process. Starting from an initial program $f^{(0)}$ produced by direct generation, IR repeatedly evaluates the current program on the training set and computes its average score J_{curr} . If the score improves over the previous iteration, the $k = 3$ worst-performing instances are selected as feedback and the LLM is prompted to produce a revised program $f^{(t+1)}$; otherwise the loop terminates and the previous program $f^{(t-1)}$, i.e., the best-performing iterate, is returned as the final policy. This mirrors the steps of Algorithm 1.

(a) Evaluation Benchmarks

(i) GRASP

The GRASP benchmark [35] (Grid-based Reasoning for Assessing Spatial Planning) is a large-scale, synthetic dataset designed to evaluate commonsense spatial reasoning in autonomous agents, with a particular emphasis on planning and action selection rather than mere interpretation of spatial descriptions. Each GRASP environment is an 11×11 two-dimensional grid comprising of cells that may be empty, contain a single unit of energy or be blocked by an obstacle. The agent is given full knowledge of the grid layout at the start of an episode and is allowed a maximum of 20 steps to act. At each step, the agent may execute movement-related actions to traverse the grid and resource-related actions to collect (TAKE) or deposit (DROP) energy. The objective is to maximize the total amount of energy successfully collected and dropped back at the starting cell within the

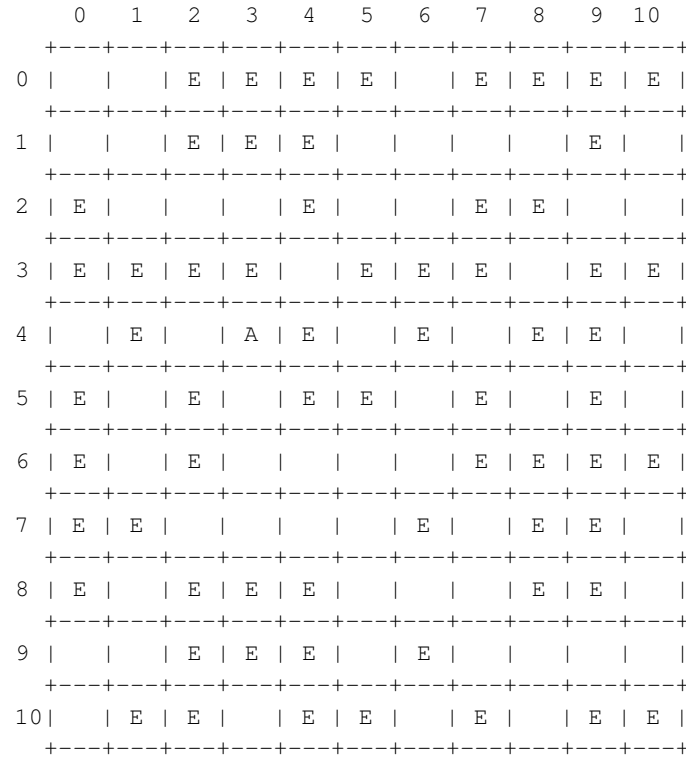


Figure 3: A sample GRASP grid instance. Each cell of the 11×11 grid is either empty, contains a single unit of energy (E), or is blocked by an obstacle (O); the agent (A) begins from a designated starting cell. The agent must plan a route that collects energy tokens and returns them to the starting cell within the step limit while minimizing movement cost.

step limit, reflecting practical scenarios in which agents must trade off exploration, exploitation, and return costs. A sample GRASP grid instance is shown in Figure 3.

The benchmark systematically varies grid configurations along three orthogonal dimensions: energy distribution, obstacle configuration, and agent starting position. Five distinct energy distribution patterns are used to induce diverse spatial structures. In the random distribution, each cell independently contains energy with a probability sampled uniformly between 0.3 and 0.7. Vertically-skewed and horizontally-skewed distributions introduce asymmetry by assigning higher energy probabilities to either one half of the grid along the vertical or horizontal axis, with complementary probabilities applied to the remaining half. The cluster distribution concentrates energy into several randomly placed 3×3 clusters, encouraging region-based reasoning. The spiral distribution generates a continuous, spiral-shaped trail of energy radiating outward from the grid center, testing the agent's ability to recognize and follow extended spatial patterns. Each grid instance uses exactly one of these five distributions. Obstacle configurations further modulate navigational complexity. In grids with obstacles, each cell can become an obstacle, rendering it impassable and unusable for energy collection.

Moreover, there are additional configurable parameters that add to the overall complexity of route optimization:

- (i) Cost per Step - Energy Expenditure for every action taken by the agent that encourages minimalist solutions.
- (ii) Carry Limit - An upper limit on the number of energy tokens allowed to be carried by the agent simultaneously.

- (iii) **Direction of Movement** - The agent movement can be limited to the four cardinal directions (UP, DOWN, LEFT, RIGHT) or can be allowed to move in also the four diagonal directions (UPRIGHT, UPLEFT, DOWNRIGHT, DOWNLEFT) increasing navigational flexibility.

By combining varied spatial layouts, explicit resource management objectives, and multiple interacting constraints within a fully observable environment that yield up to 16000 unique task instances, the benchmark probes whether an agent can transform spatial knowledge into coherent plans that balance efficiency, feasibility, and goal satisfaction.

We evaluate performance on GRASP using the *average net energy*. For a single instance, the net energy is defined as:

$$E_{\text{net}} = E_{\text{collected}} - (N_{\text{actions}} \times c_{\text{step}}), \quad (4.1)$$

where $E_{\text{collected}}$ is the total energy the agent successfully picks up and returns to its starting cell, N_{actions} is the number of actions it takes, and c_{step} is the cost per step. The metric reported for each configuration is E_{net} averaged over all of its instances. Higher net energy indicates better performance; because the action cost is subtracted, the metric can be negative when an agent expends more energy moving than it manages to collect and return.

(ii) MiniGrid

MiniGrid [7] is a collection of lightweight, configurable 2D discrete grid-world reinforcement learning environments designed to facilitate reproducible and scalable evaluation of agents on sparse, goal-oriented tasks. Each environment comprises a grid world populated with walls and interactive objects (e.g., doors, keys, boxes) and parameterizes the difficulty by grid size and object layout. The default action space for IPP experiments is limited to a discrete set of six actions: LEFT, RIGHT, MOVE, PICKUP, DROP and UNLOCK. Episodes terminate upon task completion or when a specified maximum step count is reached, and rewards are typically sparse (e.g., near-binary with a small shaping component) to reflect successful task completion. For experiments evaluating IPP on MiniGrid, 3 tasks were chosen with increasing complexity.

- (i) **Unlock**. In the Unlock task, the objective is to open a locked door that separates the agent from an exit region. The task requires the agent to locate and retrieve a key and then use an unlock action to unlock the door. A sample instance is shown in Figure 4a.
- (ii) **Door-Key**. The Door-Key environment generalizes the unlocking challenge by embedding it within a larger navigation task: the agent must first pick up a key, use it to unlock a locked door, and then navigate to a designated goal location (a green goal square). A sample instance is shown in Figure 4b.
- (iii) **Unlock-Pickup**. In the Unlock-Pickup task the agent must traverse through a locked door to reach and pick up a target box located in a separate room. A sample instance is shown in Figure 4c.

Each task adds on to the previous one thereby increasing the problem complexity. For each task, we generate 1,000 instances and measure the average reward.

(b) Prompting Strategies and Baselines

We evaluate the IPP framework using four prompting strategies namely *Direct Generation*, *Pseudocode Extension*, *Step-by-Step*, and *Iterative Refinement* which were explained in detail in Section (a). Of these, Pseudocode Extension and Step-by-Step are applied only to the GRASP benchmark, whereas Direct Generation and Iterative Refinement are evaluated across both benchmarks. This scoping follows directly from the structure of the two benchmarks. GRASP exposes a rich set of independently configurable parameters—cost per step, carry limits, obstacle density, and the set of permitted movement directions—that together span a wide and continuously tunable range of task complexity. This structure is what makes Pseudocode Extension and Step-by-Step meaningful:

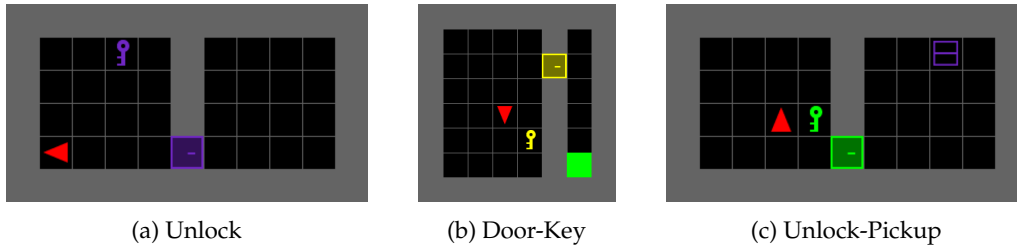


Figure 4: Sample instances of the three MiniGrid tasks used to evaluate IPP, shown in order of increasing complexity. The red triangle is the agent and grey cells are walls. (a) Unlock: the agent must retrieve the key and use it to unlock the door. (b) Door-Key: after unlocking the door, the agent must additionally navigate to the green goal square. (c) Unlock-Pickup: the agent must pass through the locked door to reach and pick up the target box in the adjacent room.

it affords a genuinely high-level algorithmic sketch that the model must adapt to many parameter regimes, and it allows a graded curriculum to be built by introducing these constraints one at a time. The MiniGrid tasks, in contrast, are defined by a small, fixed set of interaction rules rather than tunable difficulty axes. Because the intended behavior on these tasks is already captured by a compact hand-written greedy routine, providing an equivalent pseudocode prior would largely amount to supplying the solution itself, leaving little room for the model to extend or improve upon it. For the same reason, the tasks cannot be decomposed into a sequence of progressively harder sub-problems as the Step-by-Step curriculum requires. We therefore reserve these two structured strategies for GRASP, where their underlying assumptions hold. Additionally, we consider four non code based prompting strategies for plan generation. The simplest of them is *Naive Prompting*. In this setting, the model is instructed to output only a valid sequence of actions that completes the task, with no accompanying explanation, intermediate reasoning, or auxiliary text. The output format is explicitly constrained to a list of actions. This strategy tests the model’s ability to implicitly solve the planning problem in a single forward pass, without any external scaffolding or decomposition. While computationally efficient, Naive Prompting provides no mechanism for structured reasoning or error mitigation and serves as a strong baseline for evaluating the added value of more elaborate prompting schemes. To encourage more explicit reasoning, we adopt *Chain-of-Thought (CoT)* prompting [41]. Under this strategy, the model is prompted to reason through the task in natural language before producing the final action plan.

We further introduce a *2-Step CoT* variant designed to reduce the cognitive burden of generating long plans in a single pass. In this variant, planning is explicitly divided into two stages. In the first stage, the model generates a partial action sequence up to a predefined midpoint in the task horizon. The environment is then deterministically updated according to this partial plan, and the model is prompted a second time to complete the remaining portion of the plan based on the updated state. This staged decomposition enables more localized and focused reasoning at each step, reducing error propagation and allowing the model to condition later decisions on anticipated intermediate outcomes.

In addition to language-based prompting strategies, we evaluate two simple programmatic baselines that do not rely on learned reasoning or language modeling. These baselines provide reference points for interpreting task difficulty and contextualizing the performance of IPP.

The *Random* baseline uniformly samples from the set of valid actions at each step. While this agent is not expected to succeed except by chance, it establishes a clear lower bound on performance and helps characterize the sparsity and structure of the action space.

The *Greedy* baseline follows a hand-coded, myopic policy that selects actions based on immediate heuristic utility or apparent progress toward the goal. Although computationally efficient and deterministic, this policy does not account for long-term dependencies, delayed rewards, or

Table 1: Performance of LLMs on the GRASP benchmark and three MiniGrid tasks: **Unlock**, **Door-Key**, and **Unlock-Pickup**. For GRASP the metric is the average net energy collected and for MiniGrid the average reward, each reported as the mean $\pm$ the 95% confidence interval of the mean. Each model is tested using two strategies: **Direct Generation (DG)** and **Iterative Refinement (IR)**. Blue/red text indicates the percentage improvement or deterioration of IR over DG.

Model	GRASP		Unlock		Door-Key		Unlock-Pickup	
	DG	IR	DG	IR	DG	IR	DG	IR
GPT-4o	-1.53 ± 0.06	-0.45 ± 0.05 +71%	0.00 ± 0.00	0.00 ± 0.00 +0%	0.00 ± 0.00	0.00 ± 0.00 +0%	0.00 ± 0.00	0.00 ± 0.00 +0%
GPT-o1	0.94 ± 0.04	2.82 ± 0.05 +200%	0.97 ± 0.00	0.97 ± 0.00 +0%	0.97 ± 0.00	0.97 ± 0.00 +0%	0.95 ± 0.00	0.95 ± 0.00 +0%
GPT-o3-mini	2.82 ± 0.05	2.90 ± 0.04 +3%	0.90 ± 0.02	0.97 ± 0.00 +8%	0.78 ± 0.02	0.98 ± 0.00 +26%	0.00 ± 0.00	0.78 ± 0.02 + ∞ %
Claude-Sonnet-3.7	1.30 ± 0.05	2.78 ± 0.04 +114%	0.71 ± 0.03	0.90 ± 0.02 +27%	0.75 ± 0.03	0.88 ± 0.02 +17%	0.55 ± 0.03	0.94 ± 0.00 +70%
Gemini-2.5-Pro	-0.52 ± 0.03	3.22 ± 0.05 +719%	0.97 ± 0.00	0.97 ± 0.00 +0%	0.71 ± 0.03	0.90 ± 0.02 +27%	0.89 ± 0.01	0.89 ± 0.01 +0%
DeepSeek-R1	1.85 ± 0.04	1.59 ± 0.03 -14%	0.00 ± 0.00	0.97 ± 0.00 + ∞ %	0.92 ± 0.01	0.83 ± 0.02 -10%	0.86 ± 0.02	0.86 ± 0.02 +0%

irreversible mistakes, and therefore may fail on tasks requiring foresight or backtracking. As such, it provides a useful contrast to both natural language planning and code based approaches.

(c) LLMs Evaluated

We evaluate our IPP framework using six LLMs, including both general purpose LLMs and code specialized ones. Specifically, we use the following: GPT-4o [42], GPT-o1 [43], GPT-o3-mini [44], Claude-Sonnet-3.7 [45], Gemini 2.5 Pro [46], and DeepSeek-R1 [47]. All models are accessed through the OpenRouter API [48]. Code for reproducing all experiments is available [here](#).

5. Results

Table 1 shows the average energy collected in the GRASP dataset, averaged over the five energy distributions and both obstacle settings, comparing Direct Generation (DG) with Iterative Refinement (IR). Across most models (except for DeepSeek-R1), IR outperforms DG. Overall, GPT-o1 and Claude-Sonnet-3.7 show substantial gains, with IR achieving 200% and 114% improvement. Gemini-2.5-Pro exhibits the largest percentage improvement overall, turning negative DG scores (e.g., -0.52) into high positive IR scores (e.g., 3.22), with a gain of 719%. GPT-o3-mini, despite strong baseline performance in DG (e.g., 2.82 average energy), benefits only slightly from IR (a 3% gain). In contrast, GPT-4o shows limited effectiveness, with net energy scores remaining negative even after IR despite a 71% relative gain, while DeepSeek-R1 uniquely exhibits consistent performance declines (a 14% drop), suggesting potential incompatibility with the IR strategy.

Table 1 also reports the performance of the LLMs on the three MiniGrid tasks (Unlock, Door-Key, and Unlock-Pickup), comparing the DG and IR strategies by average reward. Among all models, GPT-o1 consistently shows perfect or near perfect performance under DG, leaving no room for improvement for IR. In contrast, GPT-o3-mini and Claude-Sonnet-3.7 benefit substantially from IR, especially in more complex tasks like Unlock-Pickup. GPT-o3-mini improves from total failure (0.00 reward) to strong performance (0.78 reward), while Claude-Sonnet-3.7 achieves similar gains, with a 70% increase in reward.

Other models show more varied results. Gemini-2.5-Pro displays meaningful improvement only in the Door-Key task, with static performance elsewhere. DeepSeek-R1 shows inconsistent behavior: IR dramatically increases performance in the Unlock task from 0.00 to 0.97 in reward, yet decreases performance in Door-Key and has no gain on Unlock-Pickup. On the other hand, GPT-4o fails to improve using IR across all tasks, maintaining 0 reward, indicating fundamental issues with task comprehension or execution.

Table 2 provides a comparative analysis of IR against several other prompting and programmatic baselines: CoT, 2-step CoT, Random, and Greedy, on the GRASP and MiniGrid tasks using five models.² Importantly, these approaches do not fall under the IPP framework; instead, they operate through natural language prompting and direct action generation without synthesizing executable code. For GRASP, in order to evaluate the different methods on the hardest of tasks, the instances were configured to allow eight movement directions with a carry limit of 2 and a cost per step of 0.3, with the agent starting inside the grid. On the GRASP benchmark, IR is the only one that results in a positive average energy score (0.16 ± 0.04), substantially outperforming CoT (-1.02) and 2-step CoT (-1.91). Programmatic baselines such as Greedy (-3.61) and Random (-4.55) perform even worse. Notably, IR accomplishes this with the lowest API cost among LLM based methods (8.0×10^{-4}). While all reported costs are averaged over 100 instances, it is important to note that IR generates a single final Python program that generalizes to all future instances. As a result, its cost does not grow with the number of instances evaluated. Once the program is synthesized through IR, it can be reused without invoking the model again. In contrast, CoT and 2-step CoT need to generate new action plans for each individual instance, causing their costs to scale linearly with the number of evaluations.

For the three MiniGrid tasks, IR matches or slightly exceeds the other best performing method. For instance, in the Unlock and Door-Key tasks, IR achieves a perfect or near perfect reward (0.97 and 0.98 respectively), comparable to or better than CoT and 2-step CoT, but at a substantially lower API cost (as low as 8.1×10^{-4} per instance). In the Unlock-Pickup task, IR (0.76 ± 0.07) performs better than CoT (0.48) and nearly matches 2-step CoT (0.84), again at a reduced cost. These results suggest that IR offers a better balance of performance and cost than other LLM based methods, making it practical for complex planning tasks that demand precise execution.

Figure 5 shows the performance of all six LLMs using IR across four tasks: GRASP, Unlock, Door Key, and Unlock Pickup, with the numerical results presented in Table 3. The x-axis shows the number of refinement iterations, with 0 representing direct generation and higher values indicating additional refinement steps. In the GRASP benchmark, GPT-o1 and GPT-o3-mini reach a peak energy score of 2.82 and 2.90 at iteration 1, while Claude-Sonnet-3.7 peaks at 2.78 at iteration 2 before slightly declining. Among the added models, Gemini-2.5-Pro shows the largest benefit from refinement, climbing from -0.52 at iteration 0 to a peak of 3.22 at iteration 3, whereas GPT-4o remains negative throughout (improving only from -1.53 to -0.45), and DeepSeek-R1 starts relatively strong at 1.85 but declines slightly to 1.59.

In the three MiniGrid tasks, GPT-o1 and GPT-o3-mini perform well from the beginning, often achieving near perfect scores by iteration 1. Claude-Sonnet-3.7 shows the meaningful improvement, particularly in the Unlock and Unlock Pickup tasks, where rewards increase from 0.71 to 0.90 and from 0.55 to 0.94, respectively. Compared to GRASP, performance in the MiniGrid tasks declines less often with additional refinement. The added models reinforce this picture but also expose its limits. DeepSeek-R1 improves sharply on Unlock, rising from 0.00 to 0.97, whereas GPT-4o fails to make any progress on the MiniGrid tasks, remaining at zero throughout. Refinement is not always monotonic, however: Gemini-2.5-Pro reaches a reward of 0.90 on Door-Key by iteration 2 but then regresses to 0.00 at iteration 3, showing that an additional refinement step can occasionally overwrite a previously correct program. Overall, the results show that most performance gains occur within the first one or two iterations of IR. Beyond that point, improvements tend to level off or slightly decrease.

As noted in Section (b), the Pseudocode Extension and Step-by-Step strategies are evaluated only on the GRASP benchmark. Figure 6 presents two analyses of performance on the GRASP task using all six LLMs, with corresponding numerical results provided in Table 4. On the left, it shows the comparison between the Greedy programmatic baseline and the Pseudocode Extension. GPT-o1, which performs reasonably well under the Greedy baseline, fails under Pseudocode Extension with a negative energy score. On the other hand, Claude-Sonnet-3.7 and GPT-o3-mini both benefit from this structured guidance, achieving higher energy scores with Pseudocode

²We omit GPT-o1 in the baselines experiments due to cost constraints.

Table 2: Comparison of prompting strategies and baseline methods across five LLMs on the GRASP and MiniGrid benchmarks. Within each model we report Naive Prompting, CoT, 2-step CoT, and IR; Random and Greedy are model-independent baselines shown once. For GRASP the metric is net energy collected; for MiniGrid it is reward (maximum 1.0). Each cell reports the mean and its 95% confidence interval, with the per-instance API cost in USD in parentheses. Results are averaged over 100 instances per task. For GRASP, instances use the hardest configuration: eight movement directions, a carry limit of 2, a cost per step of 0.3, and the agent starting inside the grid.

Model	Method	GRASP	Unlock Mean $\pm$ 95% CI (Cost)	Door-Key	Unlock-Pickup
Baselines	Random	-4.55 ± 0.18 (0)	0.00 ± 0.00 (0)	0.00 ± 0.00 (0)	0.00 ± 0.00 (0)
	Greedy	-3.61 ± 0.05 (0)	0.97 ± 0.00 (0)	0.98 ± 0.00 (0)	0.75 ± 0.07 (0)
GPT-4o	Naive Prompting	-1.66 ± 0.29 (\$0.0066)	0.09 ± 0.05 (\$0.0060)	0.00 ± 0.00 (\$0.0070)	0.00 ± 0.00 (\$0.0070)
	CoT	-2.12 ± 0.28 (\$0.0080)	0.11 ± 0.06 (\$0.0080)	0.00 ± 0.00 (\$0.0070)	0.00 ± 0.00 (\$0.0110)
	2-step CoT	-3.10 ± 0.30 (\$0.0146)	0.09 ± 0.05 (\$0.0090)	0.02 ± 0.03 (\$0.0180)	0.00 ± 0.00 (\$0.0160)
	IR	-0.84 ± 0.16 (\$0.0018)	0.00 ± 0.00 (\$0.0005)	0.00 ± 0.00 (\$0.0004)	0.00 ± 0.00 (\$0.0005)
GPT-o3-mini	Naive Prompting	-0.97 ± 0.13 (\$0.0018)	0.97 ± 0.00 (\$0.0010)	0.96 ± 0.03 (\$0.0020)	0.53 ± 0.09 (\$0.0020)
	CoT	-1.02 ± 0.15 (\$0.0020)	0.97 ± 0.00 (\$0.0020)	0.95 ± 0.03 (\$0.0030)	0.48 ± 0.09 (\$0.0030)
	2-step CoT	-1.91 ± 0.20 (\$0.0039)	0.97 ± 0.00 (\$0.0020)	0.97 ± 0.02 (\$0.0020)	0.84 ± 0.06 (\$0.0030)
	IR	0.16 ± 0.04 (\$0.0008)	0.97 ± 0.00 (\$0.0011)	0.98 ± 0.00 (\$0.0008)	0.76 ± 0.07 (\$0.0008)
Claude-Sonnet-3.7	Naive Prompting	-3.72 ± 0.30 (\$0.0116)	0.04 ± 0.04 (\$0.0050)	0.00 ± 0.00 (\$0.0050)	0.00 ± 0.00 (\$0.0060)
	CoT	-4.00 ± 0.30 (\$0.0115)	0.04 ± 0.04 (\$0.0050)	0.00 ± 0.00 (\$0.0060)	0.00 ± 0.00 (\$0.0060)
	2-step CoT	-3.70 ± 0.25 (\$0.0206)	0.05 ± 0.04 (\$0.0110)	0.02 ± 0.03 (\$0.0130)	0.00 ± 0.00 (\$0.0140)
	IR	-0.04 ± 0.06 (\$0.0038)	0.91 ± 0.05 (\$0.0039)	0.91 ± 0.05 (\$0.0016)	0.94 ± 0.00 (\$0.0053)
Gemini-2.5-Pro	Naive Prompting	-0.85 ± 0.17 (\$0.0103)	0.93 ± 0.04 (\$0.0020)	0.89 ± 0.06 (\$0.0020)	0.52 ± 0.09 (\$0.0040)
	CoT	-0.95 ± 0.21 (\$0.0115)	0.88 ± 0.05 (\$0.0090)	0.87 ± 0.06 (\$0.0090)	0.55 ± 0.09 (\$0.0080)
	2-step CoT	-1.11 ± 0.15 (\$0.0136)	0.90 ± 0.05 (\$0.0030)	0.76 ± 0.08 (\$0.0030)	0.54 ± 0.09 (\$0.0050)
	IR	-0.12 ± 0.07 (\$0.0019)	0.97 ± 0.00 (\$0.0017)	0.91 ± 0.05 (\$0.0045)	0.89 ± 0.04 (\$0.0026)
DeepSeek-R1	Naive Prompting	-1.39 ± 0.39 (\$0.0018)	0.87 ± 0.06 (\$0.0010)	0.85 ± 0.07 (\$0.0010)	0.71 ± 0.09 (\$0.0010)
	CoT	-1.40 ± 0.37 (\$0.0020)	0.76 ± 0.08 (\$0.0010)	0.84 ± 0.07 (\$0.0010)	0.84 ± 0.06 (\$0.0010)
	2-step CoT	-2.70 ± 0.35 (\$0.0039)	0.84 ± 0.07 (\$0.0020)	0.82 ± 0.07 (\$0.0020)	0.77 ± 0.08 (\$0.0020)
	IR	-0.65 ± 0.06 (\$0.0009)	0.97 ± 0.00 (\$0.0008)	0.87 ± 0.06 (\$0.0009)	0.87 ± 0.05 (\$0.0006)

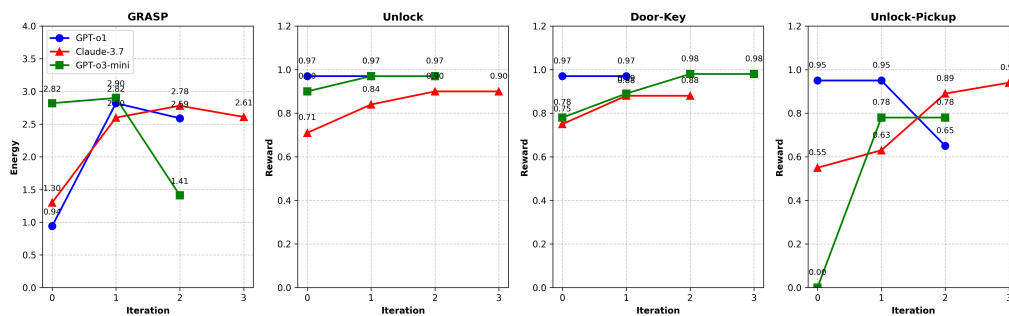


Figure 5: Performance of all six LLMs (GPT-4o, GPT-o1, GPT-o3-mini, Claude-Sonnet-3.7, Gemini-2.5-Pro, and DeepSeek-R1) across four tasks—GRASP, Unlock, Door-Key, and Unlock-Pickup—measured over successive refinement iterations. For GRASP, the metric is energy collected; for MiniGrid tasks, the metric is reward (maximum = 1.0). Iteration 0 corresponds to the initial output produced by the direct generation method.

Table 3: Performance of all six LLMs (GPT-4o, GPT-o1, GPT-o3-mini, Claude-Sonnet-3.7, Gemini-2.5-Pro, and DeepSeek-R1) across the four tasks namely *GRASP*, *Unlock*, *Door-Key*, and *Unlock-Pickup* measured over successive refinement iterations. For *GRASP*, the metric is energy collected; for MiniGrid tasks, the metric is reward (maximum = 1.0). Iteration 0 corresponds to the initial output produced by the direct generation method. Each cell is mean $\pm$ 95% confidence interval of the mean.

Dataset	Model	Iter 0	Iter 1	Iter 2	Iter 3	Iter 4	Iter 5
GRASP (Energy)	GPT-o1	0.94 $\pm$ 0.04	2.82 $\pm$ 0.05	2.59 $\pm$ 0.05	-	-	-
	Claude-Sonnet-3.7	1.30 $\pm$ 0.05	2.60 $\pm$ 0.04	2.78 $\pm$ 0.04	2.61 $\pm$ 0.05	-	-
	GPT-o3-mini	2.82 $\pm$ 0.05	2.90 $\pm$ 0.04	1.41 $\pm$ 0.06	-	-	-
	GPT-4o	-1.53 $\pm$ 0.06	-1.45 $\pm$ 0.06	-1.01 $\pm$ 0.05	-0.79 $\pm$ 0.06	-0.45 $\pm$ 0.05	-0.45 $\pm$ 0.05
	Gemini-2.5-Pro	-0.52 $\pm$ 0.03	3.10 $\pm$ 0.05	3.21 $\pm$ 0.05	3.22 $\pm$ 0.05	3.01 $\pm$ 0.05	-
	DeepSeek-R1	1.85 $\pm$ 0.04	1.59 $\pm$ 0.03	-	-	-	-
Unlock (Reward)	GPT-o1	0.97 $\pm$ 0.00	0.97 $\pm$ 0.00	0.97 $\pm$ 0.00	-	-	-
	Claude-Sonnet-3.7	0.71 $\pm$ 0.03	0.84 $\pm$ 0.02	0.90 $\pm$ 0.02	0.90 $\pm$ 0.02	-	-
	GPT-o3-mini	0.90 $\pm$ 0.02	0.90 $\pm$ 0.02	0.97 $\pm$ 0.00	0.97 $\pm$ 0.00	-	-
	GPT-4o	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	-	-	-	-
	Gemini-2.5-Pro	0.97 $\pm$ 0.00	0.97 $\pm$ 0.00	-	-	-	-
	DeepSeek-R1	0.00 $\pm$ 0.00	0.93 $\pm$ 0.01	0.97 $\pm$ 0.00	0.97 $\pm$ 0.00	-	-
Door-Key (Reward)	GPT-o1	0.97 $\pm$ 0.00	0.97 $\pm$ 0.00	0.00 $\pm$ 0.00	-	-	-
	Claude-Sonnet-3.7	0.75 $\pm$ 0.03	0.88 $\pm$ 0.02	0.88 $\pm$ 0.02	-	-	-
	GPT-o3-mini	0.78 $\pm$ 0.02	0.89 $\pm$ 0.02	0.98 $\pm$ 0.00	0.98 $\pm$ 0.00	-	-
	GPT-4o	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	-	-	-	-
	Gemini-2.5-Pro	0.71 $\pm$ 0.03	0.81 $\pm$ 0.02	0.90 $\pm$ 0.02	0.00 $\pm$ 0.00	-	-
	DeepSeek-R1	0.92 $\pm$ 0.01	0.83 $\pm$ 0.02	-	-	-	-
Unlock-Pickup (Reward)	GPT-o1	0.95 $\pm$ 0.00	0.95 $\pm$ 0.00	0.65 $\pm$ 0.01	-	-	-
	Claude-Sonnet-3.7	0.55 $\pm$ 0.03	0.63 $\pm$ 0.03	0.89 $\pm$ 0.01	0.94 $\pm$ 0.00	0.94 $\pm$ 0.00	-
	GPT-o3-mini	0.00 $\pm$ 0.00	0.78 $\pm$ 0.02	0.78 $\pm$ 0.02	-	-	-
	GPT-4o	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	-	-	-	-
	Gemini-2.5-Pro	0.89 $\pm$ 0.01	0.89 $\pm$ 0.01	-	-	-	-
	DeepSeek-R1	0.86 $\pm$ 0.02	0.86 $\pm$ 0.02	-	-	-	-

Extension, indicating that models with stronger capabilities are better at exploiting high-level pseudocode instructions. The added models sharpen this trend: DeepSeek-R1 and Gemini-2.5-Pro obtain the largest gains from Pseudocode Extension (3.28 and 2.87, up from the Greedy baseline of 0.89), whereas the weaker GPT-4o, like GPT-o1, is hurt by it and drops to -1.69. On the right, step-by-step performance is shown from step 0 to step 4 when the complexity of the *GRASP* environment is gradually added. GPT-o1 shows rapid improvement, peaking at step 2 (2.13) before dropping at step 3, then recovering at step 4. Claude-Sonnet-3.7 and GPT-o3-mini show steadier gains, with Claude-Sonnet-3.7 peaking at step 2 and declining at step 3, while GPT-o3-mini shows consistent upward trends. This step-2 local peak is shared by several models: GPT-o1, Claude-Sonnet-3.7, and DeepSeek-R1 all reach a local maximum at step 2 and then dip at step 3 before partially recovering at step 4, suggesting that each newly introduced constraint can temporarily make the problem harder and push the models toward sub-optimal solutions until they adapt. GPT-4o, by contrast, never adapts and stays near -2 across all steps, while Gemini-2.5-Pro improves almost monotonically once it clears its initially negative scores. These results suggest that incremental exposure to complexity helps models adapt, though performance can fluctuate depending on how well a model generalizes under new constraints.

Taken together, these results offer practical guidance on which strategy to use and when. Across both benchmarks, Iterative Refinement is the most reliable default: it is the only method that yields a positive average energy on the hardest *GRASP* configuration, and it matches or exceeds every non-code baseline on MiniGrid, while its cost is effectively amortized to zero because it produces a single reusable program rather than re-planning for each instance. Since almost all of its benefit arrives within the first one or two iterations, we recommend capping refinement at a small budget; additional iterations rarely help and can occasionally overwrite an already-working program. Method choice, however, interacts strongly with model capability. For strong reasoning models such as GPT-o1 and GPT-o3-mini, Direct Generation already solves the simpler tasks, so refinement mainly matters on the harder ones such as *Unlock-Pickup*, whereas the largest IR

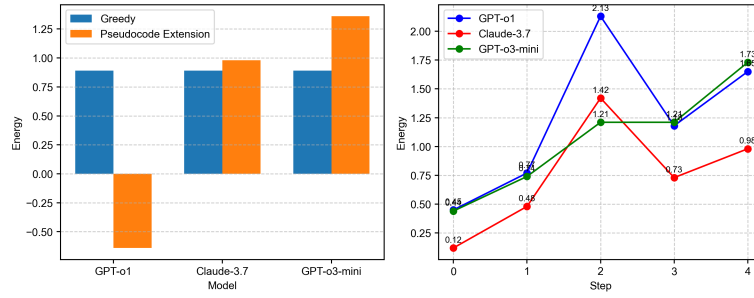


Figure 6: **Left:** Performance comparison of *Greedy* and *Pseudocode Extension* strategies on the GRASP task, evaluated across all six LLMs (GPT-4o, GPT-o1, GPT-o3-mini, Claude-Sonnet-3.7, Gemini-2.5-Pro, and DeepSeek-R1). The results highlight the performance gains enabled by pseudocode-guided reasoning. **Right:** Step-by-step performance on GRASP, where environment complexity is introduced incrementally over four steps.

Table 4: Energy Comparison by Model, Approach, and Step. Compares Greedy, Pseudocode Extension (PE), Iterative Refinement (IR) and the intermediate codes from the different steps produced by step by step methodology. Only for GRASP. Each cell is mean $\pm$ 95% confidence interval of the mean.

Model	Greedy	PE	Step 0	Step 1	Step 2	Step 3	Step 4	IR
GPT-4o	0.89 ± 0.05	-1.69 ± 0.04	-2.04 ± 0.05	-2.03 ± 0.05	-2.03 ± 0.05	-2.03 ± 0.05	-2.00 ± 0.05	-0.45 ± 0.05
GPT-o1	0.89 ± 0.05	-0.64 ± 0.03	0.45 ± 0.06	0.77 ± 0.06	2.13 ± 0.06	1.18 ± 0.03	1.65 ± 0.03	2.82 ± 0.05
GPT-o3-mini	0.89 ± 0.05	1.36 ± 0.06	0.44 ± 0.04	0.74 ± 0.04	1.21 ± 0.05	1.21 ± 0.05	1.73 ± 0.05	2.90 ± 0.04
Claude-Sonnet-3.7	0.89 ± 0.05	0.98 ± 0.05	0.12 ± 0.06	0.48 ± 0.06	1.42 ± 0.05	0.73 ± 0.02	0.98 ± 0.02	2.78 ± 0.04
Gemini-2.5-Pro	0.89 ± 0.05	2.87 ± 0.06	-0.51 ± 0.05	-0.46 ± 0.05	0.63 ± 0.05	1.62 ± 0.03	1.79 ± 0.03	3.22 ± 0.05
DeepSeek-R1	0.89 ± 0.05	3.28 ± 0.05	-1.23 ± 0.06	-1.23 ± 0.06	-0.98 ± 0.06	-1.50 ± 0.04	0.00 ± 0.00	1.85 ± 0.04

gains instead accrue to mid-capability models such as Claude-Sonnet-3.7. Pseudocode Extension is worth using when a reasonable greedy skeleton is available and the underlying model is capable enough to exploit high-level structure, as with Claude-Sonnet-3.7, GPT-o3-mini, Gemini-2.5-Pro, and DeepSeek-R1, but it can hurt models that follow the scaffold too literally, such as GPT-o1 and GPT-4o. The Step-by-Step curriculum is most useful when the target environment is complex, since incremental exposure to constraints helps models adapt, though performance can fluctuate as each new constraint is introduced. Finally, the non-code baselines, CoT and 2-step CoT, are the least attractive option whenever a code policy can be synthesized: they are weaker on the harder tasks and, unlike IR, their cost grows linearly with the number of instances evaluated. One caveat cuts across all of these choices: no strategy rescued GPT-4o, which failed to comprehend the tasks well enough for any prompting method to help, underscoring that a minimum level of model capability is a prerequisite rather than something the framework can supply.

6. Case Study

To further understand the qualitative difference between the different methods, we examine representative instances where the proposed Iterative Programmatic Planning (IPP) framework succeeds while the other baselines from Table 2 underperform or fail.

(a) GRASP

In this section, we conduct a detailed case study of a representative sample instance (grid index 9 from the Inner Spiral Block configuration) GRASP task, shown in Figure 8. This instance is particularly informative because it requires the agent to carefully balance exploration,

energy collection, and movement efficiency under explicit cost and capacity constraints. Table 5 summarizes the action sequences produced by the *GPT-o3-mini* model when solving this instance using a variety of prompting-based baselines and programmatic planning strategies.

While the baseline methods are often able to identify valid local actions such as collecting nearby energy tokens, they fail to optimize for the cost-per-step constraint. As a result, their generated solutions incur excessive movement and redundant actions, ultimately leading to negative net energy at the end of the episode. In particular, both the Naive and Chain-of-Thought (COT) prompting strategies repeatedly perform unnecessary *DROP* actions—one per carried energy token—despite the environment requiring only a single *DROP* to unload all carried items. This inefficiency, combined with unplanned navigation, significantly degrades overall performance.

Direct Generation (DG) exhibits a modest improvement over pure prompting-based approaches by producing somewhat shorter and more coherent trajectories. Nevertheless, it still fails to explicitly reason about the cumulative movement cost, resulting in avoidable energy loss that prevents it from achieving a positive final energy balance.

The limitations of large language models in maintaining consistent state representations are most evident in the 2-step COT strategy. The model loses track of the agent’s original drop-off location after several steps. This breakdown in state management causes the plan to derail midway through execution, leading an incorrect termination state with substantial energy loss.

In contrast, the Iterative Refinement (IR) strategy successfully adheres to the task constraints throughout the episode. By explicitly refining its plan and accounting for both movement costs and carry limits, IR produces a concise and goal-directed sequence of actions that avoids redundant operations and limits unnecessary exploration. As a result, it is the only method in this comparison that consistently terminates with a positive net energy collected.

To make the utility of the refinement mechanism concrete, we trace how the *GPT-o3-mini* program evolves across successive IR iterations on this instance; Figure 7 shows the token-selection core of the synthesised program at each step. The initial program (equivalent to direct generation, the DG row of Table 5) follows a short-sighted collection strategy (Figure 7(a)) and, crucially, never weighs the round-trip movement cost against the value of a token, so it terminates with negative net energy.

At the second iteration, after being shown its worst-performing grids as feedback, the model explicitly decides to “think ahead” rather than collect one token at a time. It revises the code so that the agent strings together several profitable pickups in a single excursion—accounting for its carry capacity, the number of moves needed to reach a token and return home, and the per-step cost—before making a single *DROP* back at the start cell. In the same step it adds a pruning rule that discards any move whose travel cost would exceed the single energy unit a token is worth (Figure 7(b)). This directly targets the failure mode responsible for the negative initial score, namely movement that is not justified by the energy it collects.

At the third iteration, the model refines this “greedy-chain” strategy further with two changes (Figure 7(c)). First, it opportunistically “piggy-backs” tokens that are immediately adjacent to the current cell whenever picking them up and still returning home remains feasible, so that the cost of extra moves is amortised across clustered tokens. Second, it replaces its next-token heuristic with an explicit scoring function that favours candidates lying in dense regions. Each candidate token τ is scored by a net gain

$$g(\tau) = (1 + b(\tau)) - (d_{to}(\tau) + d_{home}(\tau)) c_{step}, \quad (6.1)$$

where $b(\tau)$ is a cluster bonus equal to the number of other tokens lying within a Manhattan radius of 2 of τ , $d_{to}(\tau)$ and $d_{home}(\tau)$ are the number of steps from the agent’s current cell to τ and from τ back to the start cell respectively, c_{step} is the per-step energy cost, and the leading constant is the single unit of energy that the token itself is worth. A token is considered only if its net gain is positive, $g(\tau) > 0$, and among candidates with positive gain the one requiring the fewest total steps is chosen; the model additionally verifies that every planned route, including its *TAKE* and *DROP* actions, fits within the remaining action budget. The resulting program produces the concise,

Table 5: Comparison of prompting strategies, baseline and programmatic methods on the sample instance of the GRASP task using *GPT-o3-mini*. This instance was configured to allow all eight direction of movement for the agent with a cost per step of 0.3 and a carry limit of 2 energy tokens.

Method	Actions	Final Energy	Reason for Performance Gap
Naive Prompting	'RIGHT', 'TAKE', 'UP', 'TAKE', 'DOWNLEFT', 'DROP', 'DROP', 'DOWN', 'TAKE', 'LEFT', 'TAKE', 'UPRIGHT', 'DROP', 'DROP', 'UPRIGHT', 'RIGHT', 'TAKE', 'DOWNLEFT', 'LEFT', 'DROP'	-1	Performed a 'DROP' action for each energy token carried although only one 'DROP' is necessary to drop all the energy tokens. Failed to account for Cost per Step and therefore wasted energy in movement.
COT	'RIGHT', 'TAKE', 'UP', 'TAKE', 'DOWNLEFT', 'DROP', 'DROP', 'DOWN', 'TAKE', 'LEFT', 'TAKE', 'UPRIGHT', 'DROP', 'DROP', 'UPRIGHT', 'RIGHT', 'TAKE', 'LEFT', 'DOWNLEFT', 'DROP'	-1	Performed a 'DROP' action for each energy token carried although only one 'DROP' is necessary to drop all the energy tokens. Failed to account for Cost per Step and therefore wasted energy in movement.
2-step COT	'RIGHT', 'TAKE', 'LEFT', 'DROP', 'UPRIGHT', 'TAKE', 'DOWNLEFT', 'DROP', 'LEFT', 'LEFT', 'DOWN', 'TAKE', 'RIGHT', 'TAKE', 'UPLEFT', 'DROP', 'DROP'	-3.1	Lost track of the starting position where the energy tokens had to be dropped.
DG	'UPRIGHT', 'TAKE', 'DOWN', 'TAKE', 'LEFT', 'DROP', 'DOWNLEFT', 'TAKE', 'RIGHT', 'TAKE', 'UP', 'DROP', 'UPLEFT', 'UPLEFT', 'TAKE', 'DOWNRIGHT', 'DOWNRIGHT', 'DROP'	-0.4	Failed to account for Cost per Step and therefore wasted energy in movement.
IR	'DOWNLEFT', 'TAKE', 'RIGHT', 'TAKE', 'UP', 'DROP', 'UPRIGHT', 'TAKE', 'DOWN', 'TAKE', 'LEFT', 'DROP'	0.4	N/A

twelve-action trajectory shown in the IR row of Table 5, the only solution in this comparison to finish with positive net energy. This progression illustrates the core of the IR loop in Figure 2: at each step the model is confronted with concrete failure cases and converts them into a targeted, verifiable change in the synthesised code. This case study highlights the importance of structured, iterative planning mechanisms for enforcing global constraints and maintaining accurate state representations in sequential decision-making tasks.

(b) MiniGrid (Unlock-Pickup)

In this section, we analyze a representative sample instance of the MiniGrid Unlock-Pickup task (grid seed 28), illustrated in Figure 10. The action sequences generated by the *GPT-o3-mini* model under different methods are summarized in Table 6.

When using Naive Prompting and standard Chain-of-Thought (CoT) reasoning, the model is able to navigate through the environment and unlock the door successfully. However, both methods exhibit a fundamental failure in constraint handling: the agent attempts to pick up the box while still carrying the key. Since the environment enforces a single-object carry constraint, this oversight causes the episode to terminate unsuccessfully. These failures indicate that, despite producing seemingly coherent action sequences, the prompting-based methods struggle to consistently enforce object-interaction constraints across longer action horizons.

The 2-step CoT strategy demonstrates a partial improvement over the earlier baselines by correctly identifying the need to drop the key before attempting to pick up the box. However, this improvement is undermined by a critical spatial reasoning error. The model drops the key on the narrow cell that connects the two rooms, effectively blocking the only passage required to reach the box. As a result, the agent becomes unable to proceed, illustrating a common failure mode in which local correctness (dropping the key) is achieved at the expense of global feasibility (maintaining navigable paths).

Direct Generation (DG) further improves upon this behavior by producing a more globally coherent navigation strategy. In this case, the agent successfully reaches the vicinity of the box and drops the key in a nearby empty cell, avoiding the blockage issue observed in 2-step CoT. Despite this progress, DG still fails to complete the task due to a subtle but crucial execution error where the agent does not reorient itself to face the box before issuing the `PICKUP` action. This highlights a limitation in handling orientation-dependent actions, even when higher-level planning is largely correct.

```

1: while actions remain and uncollected tokens exist do
2:   for all uncollected tokens  $\tau$  do
3:      $C(\tau) \leftarrow d_{to}(\tau) + d_{home}(\tau) + 2$  {+2 for the TAKE and DROP actions}
4:   end for
5:    $\tau^* \leftarrow \arg \min_{\tau} C(\tau)$ 
6:   if  $\tau^*$  does not exist or its route exceeds the action budget then
7:     break
8:   end if
9:   walk to  $\tau^*$ ; TAKE; if the hand is full, walk home and DROP
10: end while

```

(a) Iteration 0 (Direct Generation): pick the nearest token by round-trip length; the per-step cost is ignored.

```

1: while actions remain and uncollected tokens exist do
2:   for all uncollected tokens  $\tau$  do
3:      $m(\tau) \leftarrow d_{to}(\tau) + d_{home}(\tau)$ 
4:     if  $m(\tau) + 2 > \text{remaining budget}$  then
5:       discard  $\tau$ 
6:     end if
7:     if  $c_{step} > 0$  and  $m(\tau) \cdot c_{step} \geq 1$  then
8:       discard  $\tau$  {new: trip costs more than the token is worth}
9:     end if
10:   end for
11:    $\tau^* \leftarrow \text{nearest profitable token}$ 
12:   chain pickups until the carry capacity is reached, then a single DROP at home
13: end while

```

(b) Iteration 1: prune unprofitable trips using the per-step cost c_{step} and chain pickups before returning home.

```

1: while actions remain and uncollected tokens exist do
2:   grab any token lying on an adjacent cell if the round trip still fits {new: piggy-back clustered tokens}
3:   for all uncollected tokens  $\tau$  whose route fits the budget do
4:      $b(\tau) \leftarrow (\# \text{tokens within Manhattan radius 2 of } \tau) - 1$  {new: cluster bonus}
5:      $g(\tau) \leftarrow (1 + b(\tau)) - (d_{to}(\tau) + d_{home}(\tau)) c_{step}$  {new: net gain}
6:   end for
7:    $\tau^* \leftarrow \arg \max_{\tau} g(\tau)$  subject to  $g(\tau^*) > 0$ 
8:   walk to  $\tau^*$ ; TAKE; return home and DROP once capacity is reached
9: end while

```

(c) Iteration 2: score tokens by cluster-aware net gain $g(\tau)$ and opportunistically collect adjacent tokens.

Figure 7: Evolution of the GRASP token-selection heuristic in the GPT-o3-mini program across iterative-refinement steps, on the instance of Figure 8. Only the token-selection core is shown; the BFS shortest-path routine, grid parsing, and action-budget bookkeeping are identical across iterations. Here $d_{to}(\tau)$ and $d_{home}(\tau)$ are the number of steps from the agent's current cell to token τ and from τ back to the start cell, and c_{step} is the per-step energy cost. Lines marked "new" are those introduced at that iteration.

Finally, the Iterative Refinement (IR) approach resolves all of the above issues and successfully completes the task. By iteratively correcting errors in earlier action sequences, IR ensures that object-handling constraints, spatial layout, and agent orientation are all simultaneously satisfied.

	0	1	2	3	4	5	6	7	8	9	10
+---+---+---+---+---+---+---+---+---+---+---+---+											
0	E	E	E	E	E	O	O	E	E		
+---+---+---+---+---+---+---+---+---+---+---+---+											
1									E		O
+---+---+---+---+---+---+---+---+---+---+---+---+											
2	E								E	O	O
+---+---+---+---+---+---+---+---+---+---+---+---+											
3	E			E	E	E	E			E	O
+---+---+---+---+---+---+---+---+---+---+---+---+											
4			E		E	E	E			E	
+---+---+---+---+---+---+---+---+---+---+---+---+											
5	O		E		E	E	O			E	
+---+---+---+---+---+---+---+---+---+---+---+---+											
6		O	E						E	E	
+---+---+---+---+---+---+---+---+---+---+---+---+											
7	E		E	E	O			A	E		O
+---+---+---+---+---+---+---+---+---+---+---+---+											
8	E			E	E	E	E	E			
+---+---+---+---+---+---+---+---+---+---+---+---+											
9	E										O
+---+---+---+---+---+---+---+---+---+---+---+---+											
10	E	E		O						O	
+---+---+---+---+---+---+---+---+---+---+---+---+											

Figure 8: GRASP grid instance (grid index 9 from the Inner Spiral Block configuration) with best solution by the Iterative Programmatic Planning (IPP) framework while other baselines from Table 2 underperform.

The resulting plan correctly drops the key in a safe location, navigates back to the box, reorients the agent appropriately, and completes the pickup without violating any constraints. This case study underscores the value of iterative, programmatic refinement in addressing compounding errors that arise from long-horizon planning, particularly in environments with strict interaction and orientation constraints.

As with GRASP, we trace how the GPT-o3-mini program changes under refinement on this instance; Figure 9 shows the relevant drop-key / pickup-box endgame. The direct-generation program (the DG row of Table 6) navigates to the box and drops the key in a nearby empty cell, but then issues the final `PICKUP` without re-orienting: because the preceding `DROP` leaves the agent facing the drop cell rather than the box, the pickup targets the wrong cell and the task fails (Figure 9(a)). After being shown this failure, the refinement makes two targeted changes (Figure 9(b)). First, it re-orientes the agent to face the box before the final `PICKUP`, which is the decisive fix for this instance. Second, it selects a drop cell whose direction differs from that of the box, so that the key is never placed on the cell the agent must occupy or pass through—removing the passage-blocking failure exhibited by weaker baselines such as 2-step CoT. Together these two edits convert the DG failure into the successful IR trajectory shown in Table 6.

7. Discussion and Limitations

Our experiments show that LLMs, when guided by a structured iterative process, can generate interpretable and reusable code based planning policies, even on problems they find initially challenging. The proposed iterative programmatic planning (IPP) framework consistently improves

- 1: **Precondition:** agent stands on an empty cell adjacent to the box, facing the box
- 2: $drop \leftarrow$ first empty cell adjacent to the agent
- 3: turn to face $drop$; **DROP** the key
- 4: **PICKUP** {bug: the DROP left the agent facing $drop$, not the box}

(a) Iteration 0 (Direct Generation).

- 1: **Precondition:** agent stands on an empty cell adjacent to the box, facing the box
- 2: $drop \leftarrow$ empty adjacent cell whose direction $\neq$ direction of the box {new}
- 3: turn to face $drop$; **DROP** the key
- 4: turn to face the box {new: re-orient before picking up}
- 5: **PICKUP**

(b) Iteration 1 (after refinement).

Figure 9: Evolution of the drop-key / pickup-box endgame in the GPT-o3-mini MiniGrid program, on the instance of Figure 10. (a) Direct generation drops the key and immediately issues **PICKUP**, leaving the agent facing the drop cell rather than the box. (b) After refinement the program chooses a drop cell that is not in the box's direction and re-orient to face the box before the pickup. The navigation, BFS, and rotation routines are identical across iterations and are omitted; lines marked "new" are those introduced by refinement.

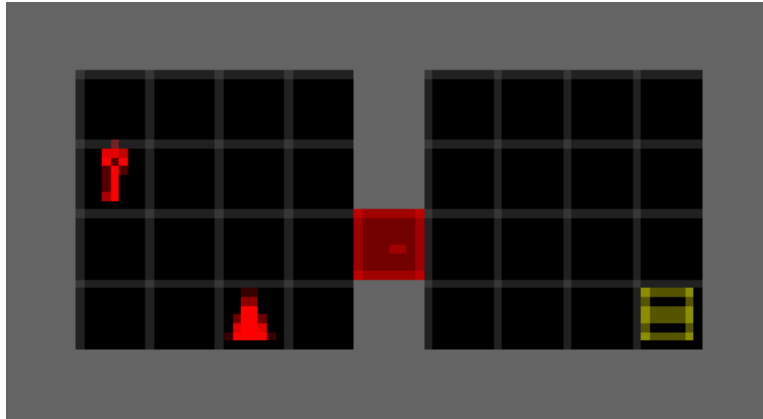


Figure 10: Sample instance (Grid seed 28) of the MiniGrid Unlock-Pickup task solved by the Iterative Programmatic Planning framework while unsolved by other baselines from Table 2.

task performance on both GRASP and MiniGrid benchmarks. Even models with a weak initial program benefit significantly from refinement, highlighting the value of execution based feedback.

By treating code as the policy representation, the agent can offload low-level responsibilities such as inventory management, environment state interpretation and management, orientation, and path planning onto deterministic Python routines. This separation of concerns allows the LLM to focus its capacity on synthesizing higher level task logic rather than repeatedly re-deriving basic operational details. As a result, the model does not need to remember or reason about aspects of the environment that can be precisely encoded and executed by the program. Instead, it can concentrate on designing control flow, conditional structures, and decision-making heuristics that constitute the actual strategy. Over successive refinement iterations, this structure provides a stable foundation on which the model can iteratively improve the policy, since deterministic code components ensure consistent behavior across executions while the LLM adjusts only the parts of the program

Table 6: Comparison of prompting strategies, baseline and programmatic methods on the sample instance (Grid seed 28) of the MiniGrid Unlock-Pickup task using *GPT-o3-mini*.

Method	Actions	Success	Reason for Failure
Naive Prompting	'MOVE', 'MOVE', 'LEFT', 'MOVE', 'PICKUP', 'LEFT', 'MOVE', 'LEFT', 'MOVE', 'MOVE', 'UNLOCK', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'RIGHT', 'PICKUP'	No	Forgot to drop the key before picking up the box
COT	'MOVE', 'MOVE', 'LEFT', 'MOVE', 'PICKUP', 'LEFT', 'MOVE', 'LEFT', 'MOVE', 'MOVE', 'UNLOCK', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'RIGHT', 'MOVE', 'LEFT', 'PICKUP'	No	Forgot to drop the key before picking up the box
2-step COT	'MOVE', 'MOVE', 'LEFT', 'MOVE', 'PICKUP', 'LEFT', 'LEFT', 'MOVE', 'MOVE', 'RIGHT', 'MOVE', 'LEFT', 'UNLOCK', 'DROP', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'RIGHT', 'PICKUP'	No	Dropped the key on the cell connecting the two rooms and hence unable to move forward
DG	'MOVE', 'MOVE', 'MOVE', 'LEFT', 'MOVE', 'MOVE', 'LEFT', 'PICKUP', 'LEFT', 'MOVE', 'RIGHT', 'MOVE', 'MOVE', 'LEFT', 'MOVE', 'MOVE', 'UNLOCK', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'RIGHT', 'RIGHT', 'RIGHT', 'DROP', 'PICKUP'	No	Navigated properly to the box and dropped the key in a nearby empty cell but forgot to turn back to face the box before picking it up
IR	'MOVE', 'LEFT', 'MOVE', 'MOVE', 'RIGHT', 'PICKUP', 'RIGHT', 'MOVE', 'MOVE', 'MOVE', 'UNLOCK', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'MOVE', 'RIGHT', 'RIGHT', 'RIGHT', 'DROP', 'RIGHT', 'RIGHT', 'PICKUP'	Yes	N/A

directly related to strategic reasoning. This leads to more reliable improvements, reduces variance introduced by free form natural language reasoning, and supports clearer attribution of errors to either the code's logic or the model's synthesis choices.

Our results demonstrate that both the structure of prompts and the choice of LLMs are critical to planning performance. While direct code generation works in simple scenarios, structured prompting strategies such as pseudocode extension and curriculum-based step-by-step prompting significantly improve outcomes on more complex tasks. For iterative refinement (IR), most performance gains emerge within the first one or two refinement steps, indicating that the process is effective and relatively computationally efficient. However, this also reveals a limitation of the approach: additional refinement iterations tend to yield diminishing returns. Once the core logic and major failure modes of the program are corrected, further improvements become increasingly incremental. This suggests that IPP is most effective at correcting high-level structural errors and missing heuristics, but may be less suited for fine-grained optimization or subtle edge cases that require highly targeted feedback or domain-specific tuning. In practice, this means that while a small number of refinement steps can substantially improve performance, extended refinement cycles may not be cost-effective and should be applied selectively.

LLMs' response to iterative refinement also varies across models. GPT-o1, Claude-Sonnet-3.7, and Gemini-2.5-Pro consistently benefit from the proposed framework, whereas DeepSeek-R1 and GPT-4o sometimes show degraded performance. This variability indicates that IPP inherits model-specific inductive biases and limitations, and that not all LLMs are equally capable of leveraging execution-based feedback to meaningfully revise previously generated code.

To understand why GPT-4o and DeepSeek-R1 do not benefit from IPP, we inspected the programs they produced at each refinement step. The two models fail for very different reasons. GPT-4o's difficulty is conceptual: its initial programs encode an incorrect model of the environment, and refinement never repairs this underlying misconception. On GRASP, it treats energy collection as a purely greedy nearest-first traversal and never weighs the reward of a token against the movement cost of reaching it, so even its converged programs spend more energy moving than they recover and stay net-negative. On the MiniGrid tasks, it repeatedly assumes the agent can interact with an object while standing on the object's own cell, rather than from an adjacent cell while facing it, so its planned paths step onto keys and doors that are in fact impassable. Successive refinements only adjust surface details such as turn selection or door traversal, and in one case leave the target object unreachable altogether, so GPT-4o never scores above zero on these tasks.

Iterative refinement can amplify competence a model already has, but it cannot supply a correct environment model that the base model lacks.

DeepSeek-R1 fails for a different reason: its programs are largely well-structured, but the model does not translate the provided failure instances into productive edits. When asked to refine, it tends to over-constrain an already-working program or add machinery that does not function. On GRASP, its refinement discards a clean collection heuristic in favour of an overly aggressive filter that rejects tokens that are only worth collecting in groups, together with inert scaffolding that is never exercised, and as a result it collects fewer tokens and regresses. On the Door-Key task, its refinement tightens the movement rules to the point of blocking cells the agent legitimately needs, lowering its reward. Rather than diagnosing the specific behaviour responsible for a failure, its revisions restrict or complicate the logic in ways only loosely related to the observed errors. This limited ability to adapt to failure feedback is consistent with the coarse feedback signal discussed above, which does not indicate which part of a program is responsible for a given failure. This behaviour may also stem from the model’s training paradigm: there have been claims in the community that Reinforcement Learning with Verifiable Rewards (RLVR) does not actually teach the model new knowledge but instead prunes the model’s answer space to a narrow subset of correct answers [49–51]. In [51], the authors note that RLVR involves a tradeoff between precision and comprehensive reasoning coverage. These observations could explain why specifically DeepSeek-R1, known to be trained using the RLVR training paradigm, underperforms when used with IPP. While GPT-o1 and GPT-o3-mini are also reasoning models, they are speculated to have been trained using an extended CoT approach with the Proximal Policy Optimization (PPO) algorithm, which still preserves the model’s ability to come up with diverse solutions for a question. Thereby, these models are able to adapt and improve their code upon seeing the failure samples, showing greater flexibility.

A major advantage of IPP is the interpretability of its code-driven policies. Unlike black-box planners or reinforcement learning agents, the generated programs are human-readable and editable, allowing experts to inspect, debug, or adapt plans (see Appendix (b) for examples). This transparency is particularly valuable in safety-critical domains such as robotics, where understanding and verifying agent behavior is essential. However, this interpretability benefit is not universal. While code offers deterministic and precise semantics, it may be less accessible to users without programming expertise. For non-technical stakeholders, natural language plans or visual representations may be easier to understand than procedural code. As a result, the interpretability advantages of IPP are currently most pronounced for users with a technical background. Bridging this gap, for example by automatically generating natural language explanations or higher-level summaries of the synthesized programs could be a natural direction for future work.

IPP is also more efficient than traditional prompting methods. While chain-of-thought prompting requires inference for each instance, IPP produces a single reusable program, reducing API costs and inference latency. Rather than competing directly with reinforcement learning, our aim is to provide a lower-cost, interpretable alternative that can solve similar planning tasks without intensive environment interaction, making it suitable for real-time or resource-constrained applications.

One other potential limitation of our framework is that it depends heavily on the language model’s ability to understand the environment and task rules. While the grid-based setup is relatively simple, models can still make mistakes in tracking the agent’s state, recognizing environment-specific entities, or following task constraints. These errors may become more pronounced in more complex environments or when the model encounters novel situations. Although iterative refinement can correct some of these issues, there is no guarantee that the final synthesized code will be correct or robust in all cases, a limitation that also applies to reinforcement learning-based policies.

Another limitation is that the feedback provided to the model during refinement is relatively coarse, typically consisting of aggregate performance metrics or a small set of failure cases. The model is not explicitly informed about which parts of the code are responsible for specific errors,

which can make the refinement process less targeted and potentially less efficient when small implementation details have a large impact on performance. We expect this to become a more serious bottleneck as environments grow in scale and horizon. In the small, fully observable grids studied here, a single aggregate score per task carries enough information for the model to infer roughly what went wrong and rewrite the relevant logic. As the state space widens and success depends on longer sequences of interdependent decisions, the same scalar becomes a progressively weaker signal: a lone reward value cannot indicate which of the many intermediate choices caused a failure, so credit assignment becomes harder and refinement is more likely to stall, thrash between competing fixes, or overfit to the handful of failure cases it happens to be shown. Settings with sparse or delayed reward, where a policy only pays off after a long correct trajectory, are especially likely to expose this weakness, as are partially observable environments in which the outcome depends on state the program never directly sees. In such regimes we would expect the coarse signal that suffices in our benchmarks to be insufficient on its own, motivating the richer, more localized feedback signals we discuss as future work.

8. Future Work

While the proposed Iterative Programmatic Planning (IPP) framework demonstrates strong empirical performance and favorable amortized cost properties on grid-based benchmarks, several important directions remain open for future investigation.

(a) Partially Observable and Noisy Environments

A primary direction to explore is extending IPP beyond fully observable, static grid worlds to environments with richer dynamics. Although grid-based domains are a foundational abstraction for many planning problems, especially in robotics, real-world settings often involve temporal constraints, stochastic transitions, and partial observability. Adapting IPP to partially observable environments would primarily require modifying the feedback provided during iterative refinement. The modification could be as simple as the LLM receiving full execution traces describing how the environment evolved in response to actions. This richer feedback could enable the model to revise heuristics and control logic to account for hidden state and observation noise. Evaluating IPP under partial observability, sensor noise, and non deterministic dynamics broaden the applicability and utility of the technique.

(b) More Complex Planning Domains

Another direction to explore is to apply IPP to a broader range of planning problems, including larger grids, more complex constraint structures, and domains that combine grid-based planning with temporal or resource constraints. Systematically stress testing IPP in domains of increasing scale and complexity would help clarify its practical limits and its comparative advantages over per instance prompting or reinforcement learning approaches. Moreover, evaluating the methodology on more complex tasks/games like Minecraft [52] would point out the ceiling of complexity of problems to which IPP is applicable.

(c) Model-Aware and Training-Aware Extensions

Performance differences across LLMs suggest that compatibility with IPP depends on the model used. As can be seen in Table 1, reasoning models, namely GPT-o1, GPT-o3-mini, and DeepSeek-R1, tend to improve on their Direct Generation code only slightly, if at all, when used with the IPP methodology in comparison to the other models. Future work could systematically study how training paradigms affect iterative code refinement and explore model-aware prompting or refinement strategies that adapt to these differences. More ambitiously, iterative refinement itself could be incorporated into post-training or fine-tuning, allowing models to explicitly learn how

to interpret failure cases and update code accordingly. Because code outputs are executable and verifiable, they provide a natural substrate for supervised fine-tuning or reinforcement learning objectives that target correctness, efficiency, and robustness.

(d) Improving the Feedback Signal and Granularity

Currently, IPP relies on aggregate performance metrics and a small set of worst-performing instances to guide refinement. Future work could explore more fine-grained feedback signals, such as automatically identifying which components of the synthesized program are responsible for failures, or annotating traces with localized error diagnostics. There is also an opportunity to incorporate expert human feedback, which could significantly enhance IPP's performance on tasks that are difficult to verify or too complex to reduce to a single numerical metric. Such signals could make refinement more targeted and reduce the number of iterations required to reach high performance. Concretely, the coarse task-level signal could be replaced with programmatically generated rich feedback, for example an execution trace that pairs each action with its outcome, or snapshots of the grid state at the points where important actions or errors occur. Such feedback requires no additional supervision, since it can be produced automatically by instrumenting the environment. A complementary direction is to introduce a critic LLM, similar to the one used in RL-GPT [52], which could take the previous code together with such an execution trace and return more guided feedback in natural language. Recent work suggests that using an LLM as a judge can be a viable source of feedback in well-defined settings such as ours [53,54], although its reliability in this planning context would need to be established empirically.

9. Related work

LLMs for Code Generation and Planning LLMs have demonstrated significant capability in code generation and other code related tasks, such as bug fixing and program testing [22,55–59]. This makes LLMs attractive for planning tasks, where structured reasoning, abstraction, and correct execution are critical. Recent frameworks use LLMs to generate and refine plans by combining high level reasoning with feedback from control or execution. [60–62]. It has been shown that LLMs can translate natural language instructions into planning formalisms like PDDL [63,64], or synthesize generalized planners that solve novel tasks via chain of thought reasoning and iterative debugging [65]. In embodied AI, LLMs have been used to generate code that serves as executable policies for robot control [28,66]. RL-GPT [52] is a framework aimed at planning and achieving a goal that requires online LLMs (fast and slow agents) to take actions, observe the effects in the environment and take further actions. The fast agent continues to refine the code based on the feedback from a critic model to complete a certain subtask. More specifically, the code specifies a list of actions (“Python-coded sub-actions”) where each snippet addresses a small slice of the task, prompting repeated LLM calls. This progress suggests the feasibility of using LLMs to produce interpretable and adaptive policies for real world tasks.

Several recent systems further demonstrate the effectiveness of treating code as a reusable policy rather than a one-off plan. Voyager [33] introduces an open-ended embodied agent that continuously generates, executes, and refines Python code skills that function as policies, storing successful behaviors in a growing skill library for lifelong learning and transfer across tasks. Similarly, GenSwarm [67] proposes an end-to-end framework in which LLMs generate interpretable, white-box code policies for multi-robot teams, enabling zero-shot generalization and scalable deployment without task-specific retraining. These works highlight the growing trend of using executable code as the primary interface between language models and environments.

Other approaches emphasize neurosymbolic structure rather than direct policy generation. WorldCoder [34], for example, uses an LLM to iteratively write and update a code-based world model that captures environment dynamics, separating model learning from planning and enabling efficient transfer across gridworld tasks. While effective, such methods focus on environment modeling rather than direct synthesis of reusable control policies.

However, much of the prior work has focused on generating action plans for specific instances, often assuming strong task priors. These methods typically emphasize plan generation over policy synthesis, and often lack mechanisms for generalization across varied problem settings. In contrast, our approach treats code as a reusable policy that can adapt to multiple instances within a task. We move beyond static plan generation and instead use LLMs to produce and improve executable programs based on observed performance. Importantly, our method requires an LLM only during the initial code generation and refinement phase. Once the code is generated, it can be reused any number of times and the LLM is no longer needed.

Iterative Code Generation Recent work on iterative code generation LLMs has shown significant progress in code quality and reliability through repeated cycles of feedback and correction [68,69]. More specifically, the SELF-REFINE framework showed that LLMs can iteratively improve their outputs by generating feedback and refining their own code, achieving up to 13% improvement [70]. Building on this idea, later approaches automate refinement using external signals such as execution feedback from test suites [71], compiler diagnostics [72], and preference guided comparisons of successful versus failed generations [73]. Additionally, some research explores multi agent and interactive refinement, where LLMs act as solvers, verifiers, or critics. This includes self play for code and test generation [74,75], critique driven updates [76], and counterexample guided refinement in inductive synthesis [77].

Iterative refinement has also been explored beyond traditional coding tasks. Reflexion [78] introduces a self-reflective loop in which LLM agents generate natural-language critiques of their own failures and use these reflections to improve future behavior, yielding strong gains in sequential decision-making and code generation without parameter updates. Program-of-Thoughts [79] prompting similarly treats generated code as an explicit reasoning and execution medium, separating high-level reasoning from computation and demonstrating improved reliability on complex reasoning tasks.

While prior work applies iterative methods to typical coding tasks like function completion or bug fixing, our approach extends this paradigm to planning. Instead of refining code for correctness, we use task level metrics like energy or reward from failures to iteratively guide program generation. Our method improves entire policy programs that can handle a range of environments, not just small code pieces. Unlike test-suite-driven refinement, our framework treats code as a control policy and refines it based on real task performance in an environment. This positions our work at the intersection of LLM-based planning, code-as-policy representations, and performance-driven iterative refinement.

10. Conclusion

In this paper, we introduced *Iterative Programmatic Planning (IPP)*, a framework that reconceptualizes planning in grid based environments as code generation. By synthesizing executable Python programs rather than raw action sequences, IPP draws on the strengths of LLMs in symbolic reasoning while addressing their weaknesses in state tracking and long horizon planning. The addition of Iterative Refinement (IR) enables LLMs to incrementally improve generated policies through feedback, producing solutions that are interpretable, reusable, and cost efficient.

Our experiments on challenging benchmarks like GRASP and MiniGrid benchmarks show that IPP consistently outperforms direct generation and non code prompting baselines, achieving substantial improvements in both task success rates and policy transparency. Furthermore, our structured prompting strategies (pseudocode extension and curriculum based synthesis) demonstrate how guided code generation can unlock significant performance gains across multiple LLMs.

Taken together, these results suggest that code driven approaches offer a practical and scalable alternative to reinforcement learning in symbolic domains. By grounding high level reasoning into executable programs, IPP provides a pathway toward building planning agents that are more transparent, generalizable, and efficient.

Data Availability Statement The code used in this study is available on [this Github repository](#) [80]. All scripts required to reproduce the results, including data processing, simulation and analysis are included.

References

1. Ajeil FH, Ibraheem IK, Azar AT, Humaidi AJ. 2020 Grid-based mobile robot path planning using aging-based ant colony optimization algorithm in static and dynamic environments. *Sensors* **20**, 1880.
2. Tripathy HK, Mishra S, Thakkar HK, Rai D. 2021 CARE: A collision-aware mobile robot navigation in grid environment using improved breadth first search. *Computers & Electrical Engineering* **94**, 107327.
3. Zhohus A, Skrynnik A, Mohanty S, Volovikova Z, Kiseleva J, Szlam A, Coté MA, Panov AI. 2022 Iglu gridworld: Simple and fast environment for embodied dialog agents. *arXiv preprint arXiv:2206.00142*.
4. Jain U, Liu IJ, Lazebnik S, Kembhavi A, Weihs L, Schwing AG. 2021 Gridtopix: Training embodied agents with minimal supervision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* pp. 15141–15151.
5. Tanzmeister G, Thomas J, Wollherr D, Buss M. 2014 Grid-based mapping and tracking in dynamic environments using a uniform evidential environment representation. In *2014 IEEE International Conference on Robotics and Automation (ICRA)* pp. 6090–6095.
6. Chevalier-Boisvert M, Bahdanau D, Lahlou S, Willems L, Saharia C, Nguyen TH, Bengio Y. 2018 Babyai: A platform to study the sample efficiency of grounded language learning. *arXiv preprint arXiv:1810.08272*.
7. Chevalier-Boisvert M, Dai B, Towers M, de Lazcano R, Willems L, Lahlou S, Pal S, Castro PS, Terry J. 2023 Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks. *CoRR* **abs/2306.13831**.
8. Eberhardinger M, Maucher J, Maghsudi S. 2023 Learning of generalizable and interpretable knowledge in grid-based reinforcement learning environments. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment* vol. 19 pp. 203–214.
9. Sukhbaatar S, Szlam A, Synnaeve G, Chintala S, Fergus R. 2015 Mazebase: A sandbox for learning from games. *arXiv preprint arXiv:1511.07401*.
10. Tamar A, Wu Y, Thomas G, Levine S, Abbeel P. 2016 Value iteration networks. *Advances in neural information processing systems* **29**.
11. Pathak D, Agrawal P, Efros AA, Darrell T. 2017 Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning* pp. 2778–2787. PMLR.
12. Cobbe K, Hesse C, Hilton J, Schulman J. 2020 Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning* pp. 2048–2056. PMLR.
13. Hart P, Nilsson N, Raphael B. 1968 A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics* **4**, 100–107.
14. Metropolis N, Ulam S. 1949 The monte carlo method. *Journal of the American statistical association* **44**, 335–341.
15. Watkins CJ, Dayan P. 1992 Q-learning. *Machine learning* **8**, 279–292.
16. Hitzler P, Eberhart A, Ebrahimi M, Sarker MK, Zhou L. 2022 Neuro-symbolic approaches in artificial intelligence. *National Science Review* **9**, nwac035.
17. Topsakal O, Edell CJ, Harper JB. 2024 Evaluating large language models with grid-based game competitions: an extensible LLM benchmark and leaderboard. *arXiv preprint arXiv:2407.07796*.
18. Kamalloo E, Dziri N, Clarke CL, Rafiei D. 2023 Evaluating open-domain question answering in the era of large language models. *arXiv preprint arXiv:2305.06984*.
19. Singhal K, Tu T, Gottweis J, Sayres R, Wulczyn E, Amin M, Hou L, Clark K, Pfohl SR, Cole-Lewis H et al.. 2025 Toward expert-level medical question answering with large language models. *Nature Medicine* pp. 1–8.
20. Qin Y, Liang X, Ye Y, Zhu K, Yan L, Lu Y, Lin Y, Cong X, Tang X, Qian B et al.. 2023 Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.
21. Qu C, Dai S, Wei X, Cai H, Wang S, Yin D, Xu J, Wen JR. 2025 Tool learning with large language models: A survey. *Frontiers of Computer Science* **19**, 198343.

22. Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Sauvestre R, Remez T et al.. 2023 Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.
23. Jiang J, Wang F, Shen J, Kim S, Kim S. 2024 A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*.
24. Valmeekam K, Marquez M, Sreedharan S, Kambhampati S. 2023 On the planning abilities of large language models-a critical investigation. *Advances in Neural Information Processing Systems* **36**, 75993–76005.
25. Cobbe K, Klimov O, Hesse C, Kim T, Schulman J. 2019 Quantifying generalization in reinforcement learning. In *International conference on machine learning* pp. 1282–1289. PMLR.
26. Korkmaz E. 2024 A survey analyzing generalization in deep reinforcement learning. *arXiv preprint arXiv:2401.02349*.
27. Cheng Z, Yu J, Xing X. 2025 A Survey on Explainable Deep Reinforcement Learning. *arXiv preprint arXiv:2502.06869*.
28. Liang J, Huang W, Xia F, Xu P, Hausman K, Ichter B, Florence P, Zeng A. 2023 Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)* pp. 9493–9500. IEEE.
29. Xu M, Huang P, Yu W, Liu S, Zhang X, Niu Y, Zhang T, Xia F, Tan J, Zhao D. 2023 Creative robot tool use with large language models. *arXiv preprint arXiv:2310.13065*.
30. Wang X, Chen Y, Yuan L, Zhang Y, Li Y, Peng H, Ji H. 2024 Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*.
31. Yang C, Wang X, Lu Y, Liu H, Le QV, Zhou D, Chen X. 2024 Large language models as optimizers. In *International Conference on Learning Representations* vol. 2024 pp. 12028–12068.
32. Huang J, Gu S, Hou L, Wu Y, Wang X, Yu H, Han J. 2023 Large language models can self-improve. In *Proceedings of the 2023 conference on empirical methods in natural language processing* pp. 1051–1068.
33. Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, Fan L, Anandkumar A. 2023 Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
34. Tang H, Key D, Ellis K. 2024 Worldcoder, a model-based llm agent: Building world models by writing code and interacting with the environment. *Advances in Neural Information Processing Systems* **37**, 70148–70212.
35. Tang Z, Kejriwal M. 2024 GRASP: A Grid-Based Benchmark for Evaluating Commonsense Spatial Reasoning. *arXiv preprint arXiv:2407.01892*.
36. Liang J, Huang W, Xia F, Xu P, Hausman K, Ichter B, Florence P, Zeng A. 2022 Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*.
37. Vemprala SH, Bonatti R, Bucker A, Kapoor A. 2024 Chatgpt for robotics: Design principles and model abilities. *Ieee Access* **12**, 55682–55696.
38. Dong Q, Li L, Dai D, Zheng C, Ma J, Li R, Xia H, Xu J, Wu Z, Liu T et al.. 2022 A survey on in-context learning. *arXiv preprint arXiv:2301.00234*.
39. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al.. 2020 Language models are few-shot learners. *Advances in neural information processing systems* **33**, 1877–1901.
40. Zhou D, Schärli N, Hou L, Wei J, Scales N, Wang X, Schuurmans D, Cui C, Bousquet O, Le Q et al.. 2022 Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*.
41. Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al.. 2022 Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* **35**, 24824–24837.
42. OpenAI. 2024a GPT-4o. .
43. OpenAI. 2024b GPT-o1. .
44. OpenAI. 2025 GPT-o3-mini. .
45. Anthropic. 2025 Claude 3.7 Sonnet. .
46. DeepMind G. 2025 Gemini 2.5 Pro. .
47. AI D. 2025 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. .
48. OpenRouter. 2024 OpenRouter. <https://openrouter.ai>. <https://openrouter.ai>.
49. Yue Y, Chen Z, Lu R, Zhao A, Wang Z, Song S, Huang G. 2025 Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model?. *arXiv preprint arXiv:2504.13837*.
50. Chen X, Li T, Zou D. 2025 On the Mechanism of Reasoning Pattern Selection in Reinforcement Learning for Language Models. *arXiv preprint arXiv:2506.04695*.

51. Wu F, Xuan W, Lu X, Liu M, Dong Y, Harchaoui Z, Choi Y. 2025 The invisible leash: Why rlvr may or may not escape its origin. *arXiv preprint arXiv:2507.14843*.
52. Liu S, Yuan H, Hu M, Li Y, Chen Y, Liu S, Lu Z, Jia J. 2024 RL-gpt: Integrating reinforcement learning and code-as-policy. *Advances in Neural Information Processing Systems* **37**, 28430–28459.
53. Li D, Jiang B, Huang L, Beigi A, Zhao C, Tan Z, Bhattacharjee A, Jiang Y, Chen C, Wu T et al.. 2025 From generation to judgment: Opportunities and challenges of llm-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* pp. 2757–2791.
54. Gu J, Jiang X, Shi Z, Tan H, Zhai X, Xu C, Li W, Shen Y, Ma S, Liu H et al.. 2026 A survey on llm-as-a-judge. *The Innovation* **7**.
55. Chen M, Tworek J, Jun H, Yuan Q, Pinto HPDO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G et al.. 2021 Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
56. Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, Barham P, Chung HW, Sutton C, Gehrmann S et al.. 2023 Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* **24**, 1–113.
57. Jiang N, Liu K, Lutellier T, Tan L. 2023 Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* pp. 1430–1442. IEEE.
58. Kang S, Yoon J, Yoo S. 2023 Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* pp. 2312–2323. IEEE.
59. Wang Y, Le H, Gotmare AD, Bui ND, Li J, Hoi SC. 2023 Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*.
60. Ajay A, Han S, Du Y, Li S, Gupta A, Jaakkola T, Tenenbaum J, Kaelbling L, Srivastava A, Agrawal P. 2023 Compositional foundation models for hierarchical planning. *Advances in Neural Information Processing Systems* **36**, 22304–22325.
61. Wang S, Han M, Jiao Z, Zhang Z, Wu YN, Zhu SC, Liu H. 2024 Llm³: Large language model-based task and motion planning with motion failure reasoning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* pp. 12086–12092. IEEE.
62. Meng Y, Chen F, Chen Y, Fan C. 2025 AuDeRe: Automated Strategy Decision and Realization in Robot Planning and Control via LLMs. *arXiv preprint arXiv:2504.03015*.
63. Oswald J, Srinivas K, Kokel H, Lee J, Katz M, Sohrabi S. 2024 Large language models as planning domain generators. In *Proceedings of the International Conference on Automated Planning and Scheduling* vol. 34 pp. 423–431.
64. Smirnov P, Joubin F, Ceravola A, Gienger M. 2024 Generating consistent PDDL domains with Large Language Models. *arXiv preprint arXiv:2404.07751*.
65. Silver T, Dan S, Srinivas K, Tenenbaum JB, Kaelbling L, Katz M. 2024 Generalized planning in pddl domains with pretrained large language models. In *Proceedings of the AAAI conference on artificial intelligence* vol. 38 pp. 20256–20264.
66. Luo H, Wu J, Liu J, Antwi-Afari MF. 2024 Large language model-based code generation for the control of construction assembly robots: A hierarchical generation approach. *Developments in the Built Environment* **19**, 100488.
67. Ji W, Chen H, Chen M, Zhu G, Xu L, Groß R, Zhou R, Cao M, Zhao S. 2026 Genswarm: Scalable multi-robot code-policy generation and deployment via language models. *npj Robotics* **4**, 5.
68. Chen X, Lin M, Schärli N, Zhou D. 2023 Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*.
69. Jiang N, Li X, Wang S, Zhou Q, Hossain SB, Ray B, Kumar V, Ma X, Deoras A. 2024 Training LLMs to better self-debug and explain code. *arXiv preprint arXiv:2405.18649*.
70. Madaan A, Tandon N, Gupta P, Hallinan S, Gao L, Wiegrefe S, Alon U, Dziri N, Prabhume S, Yang Y et al.. 2023 Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* **36**, 46534–46594.
71. Ding Y, Min MJ, Kaiser G, Ray B. 2024 Cycle: Learning to self-refine the code generation. *Proceedings of the ACM on Programming Languages* **8**, 392–418.
72. Bi Z, Wan Y, Wang Z, Zhang H, Guan B, Lu F, Zhang Z, Sui Y, Jin H, Shi X. 2024 Iterative refinement of project-level code context for precise code generation with compiler feedback. *arXiv preprint arXiv:2403.16792*.
73. Tao L, Chen X, Yu T, Mai T, Rossi R, Li Y, Mitra S. 2024 Codelutra: Boosting llm code generation via preference-guided refinement. *arXiv preprint arXiv:2411.05199*.

74. Adnan M, Xu Z, Kuhn CC. 2025 Large Language Model Guided Self-Debugging Code Generation. *arXiv preprint arXiv:2502.02928*.
75. Lin Z, Shen S, Shang J, Weston J, Nie Y. 2025 Learning to Solve and Verify: A Self-Play Framework for Code and Test Generation. *arXiv preprint arXiv:2502.14948*.
76. Zhou C, Zhang X, Song D, Chen X, Gu W, Ma H, Tian Y, Zhang M, Hu L. 2025 RefineCoder: Iterative Improving of Large Language Models via Adaptive Critique Refinement for Code Generation. *arXiv preprint arXiv:2502.09183*.
77. Wei A, Suresh T, Cao J, Kannan N, Wu Y, Yan K, Teixeira TS, Wang K, Aiken A. 2025 CodeARC: Benchmarking Reasoning Capabilities of LLM Agents for Inductive Program Synthesis. *arXiv preprint arXiv:2503.23145*.
78. Shinn N, Cassano F, Gopinath A, Narasimhan K, Yao S. 2023 Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* **36**, 8634–8652.
79. Chen W, Ma X, Wang X, Cohen WW. 2022 Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.
80. Aravindan AV. 2024 LLM-Planning-CodeGen. <https://github.com/Mystic-Slice/LLM-Planning-CodeGen>.